VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE & ENGINEERING



WEB APPLICATION DEVELOPMENT
IT093IU

PROJECT REPORT

CYBERSECURITY ONLINE LEARNING PLATFORM

Submitted by
Nguyễn Quốc Tuấn - ITITIU22177

Date Submitted:	21/10/2023
Date Performed:	3/5/2025
Course Instructor:	Dr. Nguyen Van Sinh
	MSc Tran Khai Minh

Table of Contents

List of Figures v

1 Introduction 1

2 Project Overview 1

2.1 Theory Lessons: 1

2.2 Interactive Labs: 1

2.3 Admin Dashboard: 1

3 Requirement analysis 2

3.1 Functional Requirement 2

a. User Management 2

b. Course Management 2

c. Learning Expierence 2

d. Interactive Labs 2

e. Administration Dashboard 2

3.2 Non-Functional Requirements 2

a. Performance 2

b. Security 3

c. Usability 3

d. Compatibility 3

3.3 Technical Requirements 3

a. Development Technologies 3

b. Potential Future Features 3

4 Design 4

4.1 Backend Architecture Design 4

a. System Architecture 4

b. Key Components Design 4

c. Controller Layer 5

d. Middleware Components 5

e. API Design 6

4.2 Frontend Design 9

a. Component Design 9

b. State Management 10

c. Routing Structure 10
d. API Integration 10
e. Performance Considerations 10
f. Service Layer Architecture 10
4.3 Database design 11
a. Entity-Relationship Diagram (ERD) 11
4.4 Use Case Diagram 13
a. Sequence Diagram 1: Log in 14
b. Sequence Diagram 2: Register 15
c. Sequence Diagram 3: Browsing Lesson 16
d. Sequence Diagram 4: Completing Lesson 17
e. Sequence Diagram 5: Browsing lab 18
f. Sequence Diagram: Starting a lab 19
g. Sequence Diagram 7: Submiting a flag 20
h. Sequence Diagram 8: Stopping a lab 21
i. Sequence Diagram 9: Managing Account 22
j. Sequence Diagram 10: Create Lesson 23
k. Sequence Diagram 11: Edit Lesson 23
l. Sequence Diagram 12: Reset password 24
5 Implementation 25
5.1 Authenticate Section: 25
a. Login: 25
b. Register Account 26
c. Reset Password 26
5.2 Labs Section: 27
a. Browsing lab: 27
b. Selecting and Finishing Lab 28
5.3 Lessons Section: 30
a. Browsing Lessons: 30
b. Starting Lessons 30
5.4 Account Section 33
a. Updating account 33

5.5 Admin Section **33**
a. Creating Lesson: **34**
b. Editing Lessons **35**
6 Discussion and conclusion **37**
6.1 Challenges Encountered **37**
6.2 Future Work **37**
a. AI-Powered Personalized Feedback Systems **37**
b. Advanced Laboratory Environments **38**
References **39**

List of Figures

Figure 1. ERD 11
Figure 2. Use Case Diagram 13
Figure 3.Login Sequence Diagram 14
Figure 4. Register Sequence Diagram 15
Figure 5. Browsing Lesson Sequence Diagram 16
Figure 6. Finish Lesson Sequence Diagram 17
Figure 7. Browsing Labs Sequence Diagram 18
Figure 8. Starting Labs Sequence Diagram 19
Figure 9. Submitting Flags Sequence Diagram 20
Figure 10. Stopping Labs Sequence Diagram 21
Figure 11. Managing Account Sequence Diagram 22
Figure 12. Creating Lessons Sequence Diagram 23
Figure 13. Editing Lessons Sequence Diagram 23
Figure 14. Reset Password Sequence Diagram 24
Figure 15. Login Form 25
Figure 16. Login Successful 25
Figure 17. Register Form 26
Figure 18. Register Successful 26
Figure 19. Reset Password Email 27
Figure 20. Reset Password Form 27
Figure 21. Labs Section 28
Figure 22. A Lab Card 28
Figure 23. Starting a Lab 29
Figure 24. Submitting Flag 29
Figure 25. A completed Lab 30
Figure 26. Lessons Section 30
Figure 27. An example page 31
Figure 28. A Unfinished Lesson 31
Figure 29. Completing a Lesson 32

Figure 30. A completed Lesson 32
Figure 31.Editting Account Form 33
Figure 32. List of Lessons 33
Figure 33. General Information of a Lesson 34
Figure 34. Pages of Lesson Section 34
Figure 35. Creating a Lesson successful 35
Figure 36. Editting Lesson Section 35
Figure 37. Editting Pages 36
Figure 38. Editting Lesson successful 36
Figure 39. Editted Lesson 37

1 Introduction

- This report presents the design, development, and features of TS Academy, a cutting-edge online platform focused on delivering comprehensive cybersecurity education. As digital threats continue to evolve rapidly, the need for accessible, practical cybersecurity training has never been greater. TS Academy meets this demand by offering an engaging learning environment that blends foundational theory with hands-on practice.
- Catering to learners of all skill levels, from novices exploring digital security fundamentals to experienced users sharpening advanced techniques, TS Academy employs a dual learning approach. Its robust theoretical courses provide a deep understanding of cybersecurity principles, concepts, and best practices, while interactive labs enable learners to apply their knowledge in realistic, simulated environments.

2 Project Overview

TS Academy is built on three core pillars:

2.1 Theory Lessons:

The platform delivers structured, engaging lessons on essential cybersecurity concepts, principles, and best practices. These lessons provide learners with a strong knowledge foundation to comprehend digital threats and defense mechanisms, designed to be accessible and relevant for users at any stage of their learning journey.

2.2 Interactive Labs:

To complement theoretical learning, interactive labs offer a hands-on environment where users can put their knowledge into practice. These labs simulate real-world cybersecurity scenarios, allowing learners to test defensive strategies, detect vulnerabilities, and build practical skills in a secure, controlled setting, reinforcing their theoretical understanding.

2.3 Admin Dashboard:

A powerful admin dashboard equips platform administrators with tools to manage operations efficiently. Features include user management, content creation and organization for lessons and labs, progress tracking, and potential report generation, ensuring seamless maintenance and oversight of the learning ecosystem.

3 Requirement analysis

3.1 Functional Requirement

a. User Management

- User registration and authentication system
- Role-based access control (student,administrator)
- User profile management with customization options

b. Course Management

- Course creation and publishing
- Content organization by categories, subjects, and difficulty levels

c. Learning Expierence

- Interactive lesson modules with progress tracking

d. Interactive Labs

- Hands-on virtual lab environments for practical exercises
- Sandbox environments for experimentation
- Lab exercise tracking and progress monitoring
- Simulated environments for real-world scenario practice
- Lab result submission and feedback system

e. Administration Dashboard

- User activity monitoring and engagement metrics
- Content management and moderation tools

3.2 Non-Functional Requirements

a. Performance

- Page load times under 3 seconds
- Efficient database queries to handle large datasets

- Scalable architecture to accommodate growth

b. Security

- End-to-end encryption for sensitive data

c. Usability

- Intuitive, user-friendly interface design
- Consistent navigation and interface patterns
- Clear error messages

d. Compatibility

- Support for major browsers (Chrome, Firefox, Safari, Edge)

3.3 Technical Requirements

a. Development Technologies

- Front-end: Modern JavaScript framework (React)
- Back-end: Scalable server-side technology (Node.js)
- Database: Relational (MySQL)
- Version control system for collaborative development(Github)

b. Potential Future Features

- AI-powered learning path recommendations
- Augmented reality/virtual reality learning experiences
- Advanced analytics with predictive capabilities
- Blockchain-based credential verification
- Marketplace for educational resources
- Peer-to-peer tutoring network
- Content localization and internationalization

4 Design

4.1 Backend Architecture Design

The backend architecture of our online learning platform follows a modular, service-oriented approach built on Node.js. This design ensures scalability, maintainability, and separation of concerns through well-defined components.

a. System Architecture

The backend implements a classic MVC (Model-View-Controller) pattern with additional middleware components to handle cross-cutting concerns. The system is organized into logical modules that encapsulate specific functionality domains:

- **Models**: Data schemas and business logic
- **Controllers**: Request handling and response formatting
- **Routes**: API endpoint definitions and request routing
- **Middleware**: Cross-cutting concerns like authentication
- **Config**: Environment and application configuration
- **Utils**: Shared utility functions
- **Content**: Security-focused content resources

b. Key Components Design

4.1.b.1 Data Models

The platform's data layer is structured around these primary entities:

4.1.b.2 User Model

- Manages user profiles, authentication credentials, and role-based permissions
- Supports multiple user types (student, administrator)
- Handles user-specific settings and preferences

4.1.b.3 Lesson Model

- Represents structured learning content organized into courses
- Supports versioning and content management
- Includes metadata for categorization and discovery

4.1.b.4 LessonPage Model

- Represents individual content pages within lessons
- Supports multiple content formats (text, interactive elements)
- Tracks content revisions and publishing status

4.1.b.5 UserLessonProgress Model

- Tracks individual user progress through course materials
- Records completion status

4.1.b.6 Lab Model

- Defines interactive environments for hands-on practice
- Specifies resource requirements and configuration parameters
- Supports different scenario types and difficulty levels

c. Controller Layer

Controllers implement the application's business logic and mediate between the data models and API endpoints:

4.1.c.1 Authentication Controller

- Manages user registration, login, and session management
- Implements security measures including password hashing and token validation

4.1.c.2 User Controller

- Manages user profile operations and settings
- Handles role-based permission checks
- Processes user activity and progress data
- Handles credential updates

4.1.c.3 Lesson Controller

- Delivers educational content based on user progress
- Manages content sequencing and prerequisite enforcement

4.1.c.4 Lab Controller

- Manages the lifecycle of interactive lab environments
- Processes user actions within lab scenarios
- Evaluates lab exercise completion and provides guidance

4.1.c.5 Docker Controller

- Handles container provisioning and management for isolated lab environments
- Manages resource allocation and cleanup
- Ensures security boundaries between user environments

d. Middleware Components

The middleware layer handles cross-cutting concerns:

4.1.d.1 Authentication Middleware

- Validates user session tokens
- Implements role-based access control for protected resources
- Maintains security context throughout request processing

4.1.d.2 Utility Services

Shared utilities provide common functionality:

4.1.d.3 JWT Utilities

- Generates and validates JSON Web Tokens for authentication
- Implements token refresh and expiration logic
- Provides security context for authorized operations

4.1.d.4 File Utilities

- Handles secure file operations
- Manages content uploads and storage
- Implements access control for protected resources

e. API Design

Our online learning platform implements a comprehensive RESTful API architecture organized by functional domains. The API is designed with security, performance, and maintainability as key priorities.

4.1.e.1 API Structure and Implementation

The API is built using Express.js with the following core components:

4.1.e.1.1 Authentication API

The authentication API handles user identity verification and session management:

```
router.post("/login", authController.login);
router.post("/register", authController.register);
router.get("/check-auth", checkAuth, authController.getAuthStatus);
router.post("/logout", authController.logout);
```

These endpoints manage:

- User registration with credential validation
- Secure authentication with JWT token issuance
- Session validation for protected resources
- Secure logout and session termination

4.1.e.1.2 User Management API

The user management API provides access to user profiles and account settings:

```
router.get("/home", (req, res) => {
  res.json({ message: "valid", user_name: req.user_name });
});
router.get("/account", userController.getUser);
router.post("/update-account", userController.updateUser);
```

These endpoints handle:

- User profile retrieval
- Account settings updates
- User preference management

4.1.e.1.3 Learning Content API

The learning content API manages educational resources and user progress:

```
router.get("/", lessonController.getAllLessons);
router.get("/:id", lessonController.getLessonById);
router.post("/create", lessonController.createLesson);
router.put("/:id", lessonController.updateLesson);
router.delete("/:id", lessonController.deleteLesson);
```

```
router.get("/:lessonId/pages/:pageNumber", lessonController.getLessonPage);
router.get("/:lessonId/pages", lessonController.getAllLessonPages);
router.post("/:lessonId/pages", lessonController.createLessonPage);
router.put("/pages/:pageId", lessonController.updateLessonPage);
router.delete("/pages/:pageId", lessonController.deleteLessonPage);
```

```
router.post("/progress/:lessonId", lessonController.updateUserProgress);
router.get("/user/progress", lessonController.getUserLessonsWithProgress);
```

These endpoints provide:

- Comprehensive lesson CRUD operations
- Granular access to lesson content pages
- User progress tracking and management
- Content organization and sequencing

4.1.e.1.4 Interactive Lab API

The lab API handles the creation and management of hands-on learning environments:

```
router.get("/", labController.labsWithStatus);
router.get("/verify-flag", labController.verifyFlag);
router.post(
  "/create-container",
  labController.getLabName,
  createContainerController
);
router.post("/getlabname", labController.getLabName);
router.post("/delete-container", deleteContainer);
```

These endpoints manage:

- Lab environment discovery and status checking
- Container-based lab environment provisioning
- Challenge verification and completion tracking
- Resource cleanup and management

4.1.e.1.5 API Security Features

The API incorporates several security measures:

4.1.e.1.5.1 Authentication Middleware:

All protected routes use the checkAuth middleware to validate user sessions:

```
app.use("/api/route", checkAuth, indexRoutes);
app.use("/api/labs", checkAuth, labRoutes);
app.use("/api/lessons", checkAuth, lessonRoutes);
```

4.1.e.1.5.2 CORS Protection:

Strict cross-origin resource sharing policies:

```
app.use(
  cors({
    origin: "http://localhost:4000",
    methods: ["GET", "POST", "PUT", "DELETE"],
    credentials: true,
  })
);
```

4.1.e.1.5.3 Error Handling:

Consistent error responses for invalid routes:

```
app.use((req, res) => {
  res.status(404).json({ message: "Route not found" });
});
```

4.2 Frontend Design

The frontend of our online learning platform is built as a single-page application (SPA) using React.js, providing a responsive and dynamic user experience. This is a well-structured React application built with TypeScript and Vite, organized with a feature-based approach.

a. Component Design

The frontend implements a component-based architecture with clear separation of concerns:

4.2.a.1 Core Components

- **App Component**: Serves as the application root, managing global state and routing
- **Navigation Component**: Provides consistent navigation across the platform
- **Authentication Components**: Handle user login, registration, and session management
- **Protected Route Component**: Ensures authenticated access to restricted areas

4.2.a.2 Feature Components

4.2.a.2.1 Lesson Components

- **LessonView**: Displays lesson content to users with navigation controls
- **LessonEditor**: Provides content creation and editing interface for admin
- **LessonCreate**: Facilitates the creation of new educational content
- **LessonCards**: Displays available lessons with progress indicators

4.2.a.2.2 Lab Components

- **Lab**: Container component for interactive lab environments
- **LabCards**: Selection interface for available lab exercises

4.2.a.2.3 Admin Interface

- Administrative dashboard for platform management
- User, lesson, and lab management controls

b. State Management

The application uses a combination of:

- Local component state with React's useState hook
- Context API for global state management (authentication, user preferences)
- Prop drilling for component communication within feature modules

c. Routing Structure

The frontend implements client-side routing with React Router:

- Public routes (Home, About, Login)
- Protected routes requiring authentication (Account, Lessons, Labs, Admin)
- Nested routes for complex features (lesson pages, lab exercises)
- 404 handling for invalid routes

d. API Integration

The frontend communicates with the backend through:

- RESTful API calls to the server endpoints
- Authentication token management
- Error handling and loading states

e. Performance Considerations

Several strategies are employed to optimize performance:

- Code splitting for route-based lazy loading
- Asset optimization for efficient delivery
- Memoization for expensive computations
- Efficient rendering of lists and dynamic content

f. Service Layer Architecture

The application implements a robust service layer that acts as an intermediary between the UI components and backend APIs:

4.2.f.1 Core Services

- **authService**: Manages authentication flows, token handling, and user session persistence
- **accountService**: Handles user profile operations, preferences, and account management
- **adminService**: Provides administrative functionality and system configuration operations

4.2.f.2 Feature Services

- **lessonService**: Facilitates CRUD operations for lessons, content retrieval, and progress tracking
- **labService**: Manages interactive lab environments, exercise submissions, and progress evaluation

4.2.f.3 Service Architecture Patterns

- Implementation of the repository pattern for data access abstraction
- Consistent error handling and response normalization
- Caching strategies for frequently accessed data
- Request interceptors for authentication header injection
- Response transformation for component-ready data structures

4.3 Database design

a. Entity-Relationship Diagram (ERD)

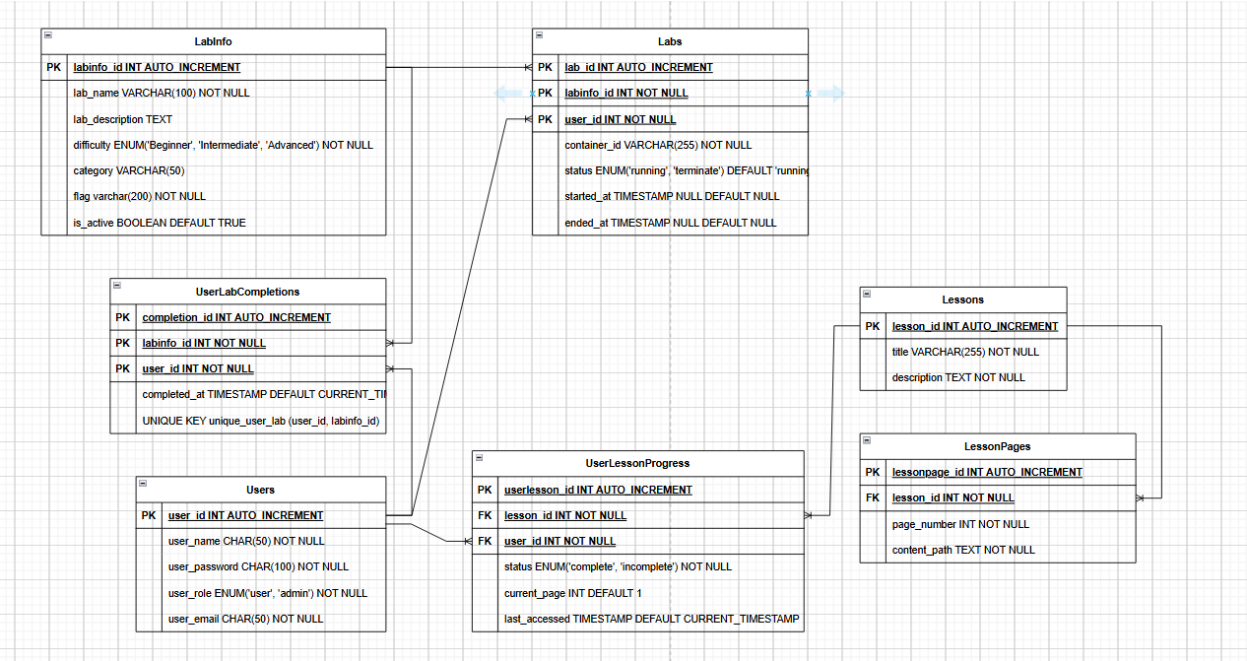


Figure 1. ERD

4.3.a.1 Users table

Users(<u>user_id</u>, user_name, user_password, user_role, user_email)

4.3.a.2 LabInfo table

LabInfo(<u>labinfo_id</u>, lab_name, lab_description, difficulty, category, flag, is_active)

4.3.a.3 Labs table

Labs(<u>lab_id</u>, user_id, labinfo_id, container_id, status, started_at, ended_at)

from Labs.user_id to Users.user_id

from Labs.labinfo_id to LabInfo.labinfo_id

4.3.a.4 UserLabCompletions table

UserLabCompletions(completion_id, user_id, labinfo_id, completed_at)

from UserLabCompletions.user_id to Users.user_id

from UserLabCompletions.labinfo_id to LabInfo.labinfo_id

4.3.a.5 Lessons table

Lessons(lesson_id, title, description)

4.3.a.6 LessonPages table

LessonPages(lessonpage_id, lesson_id, page_number, content_path)

from LessonPages.lesson_id to Lessons.lesson_id

4.3.a.7 UserLessonProgress table

UserLessonProgress(userlesson_id, user_id, lesson_id, status, current_page, last_accessed)

from UserLessonProgress.user_id to Users.user_id

from UserLessonProgress.lesson_id to Lessons.lesson_id

4.4 Use Case Diagram

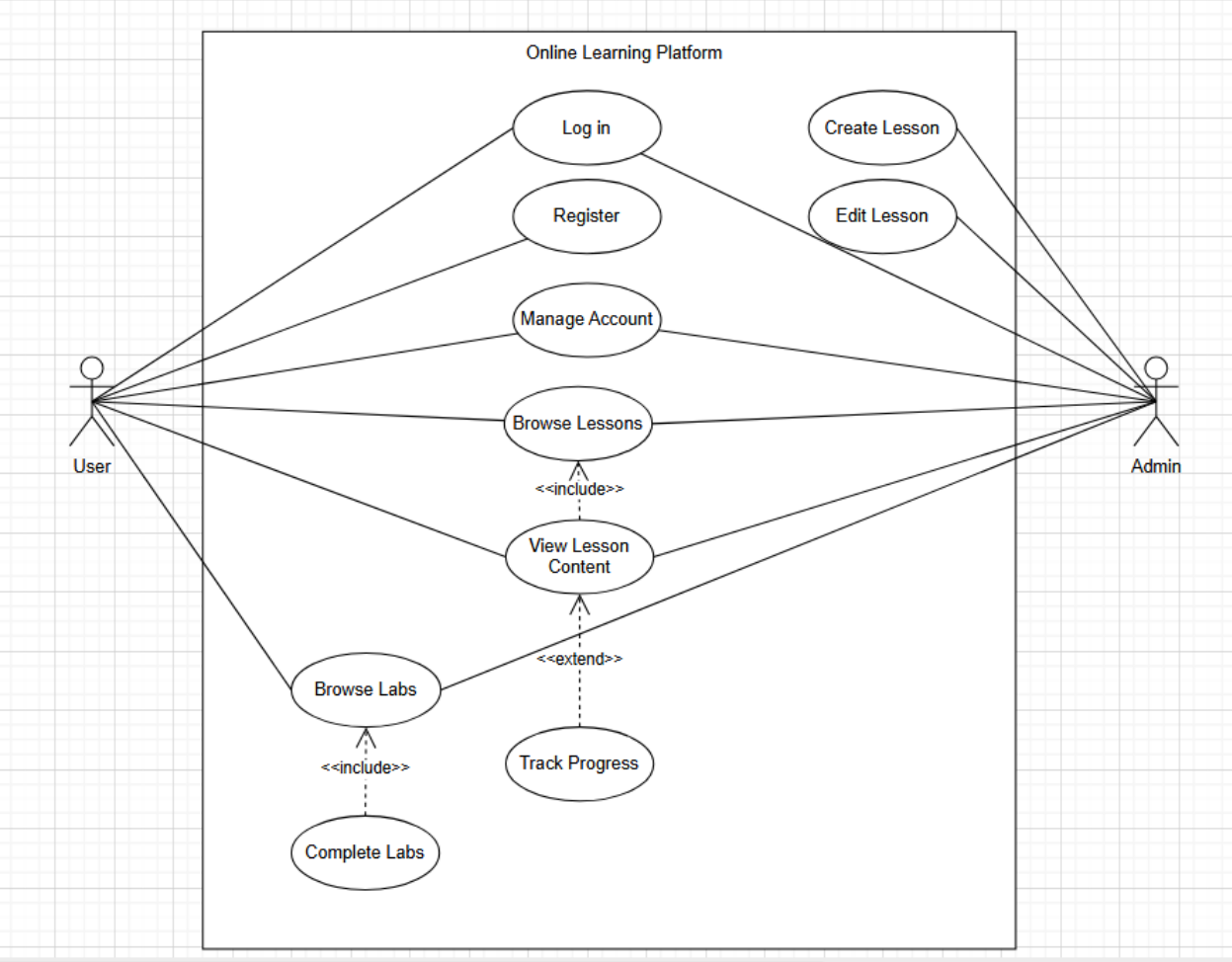


Figure 2. Use Case Diagram

a. Sequence Diagram 1: Log in

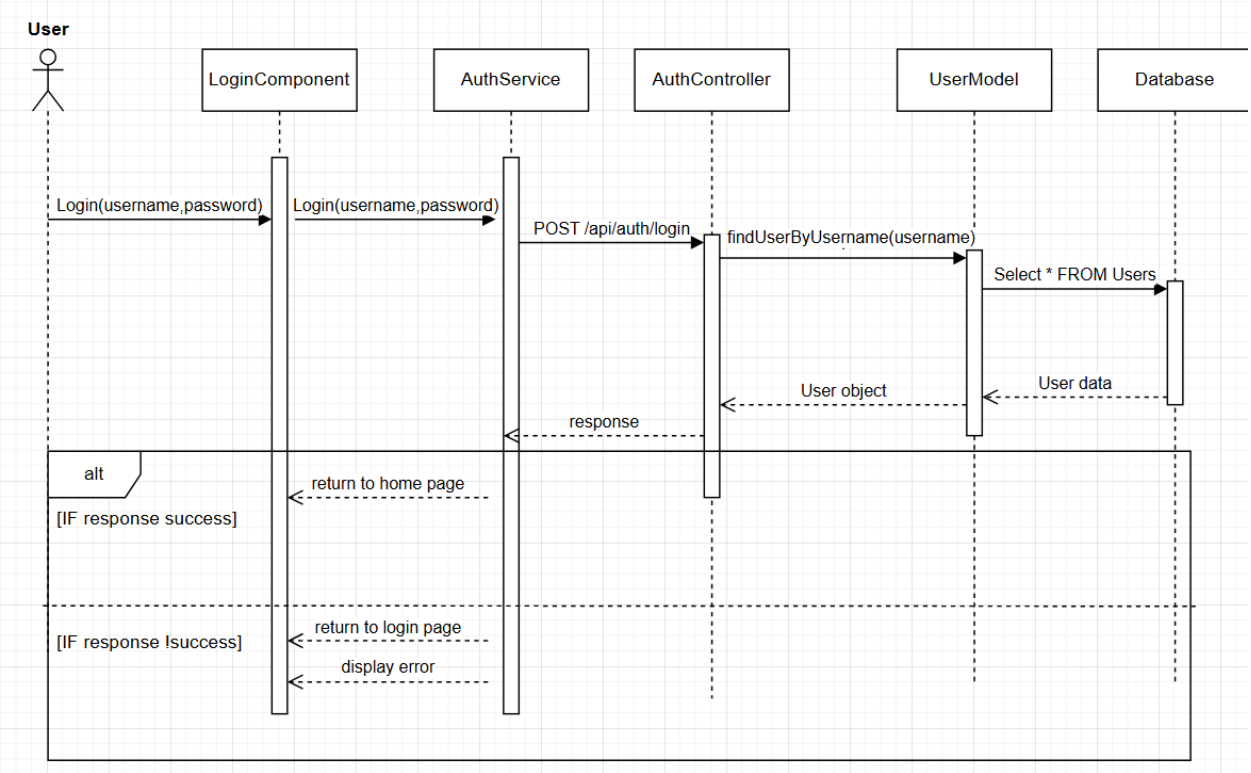


Figure 3.Login Sequence Diagram

b. Sequence Diagram 2: Register

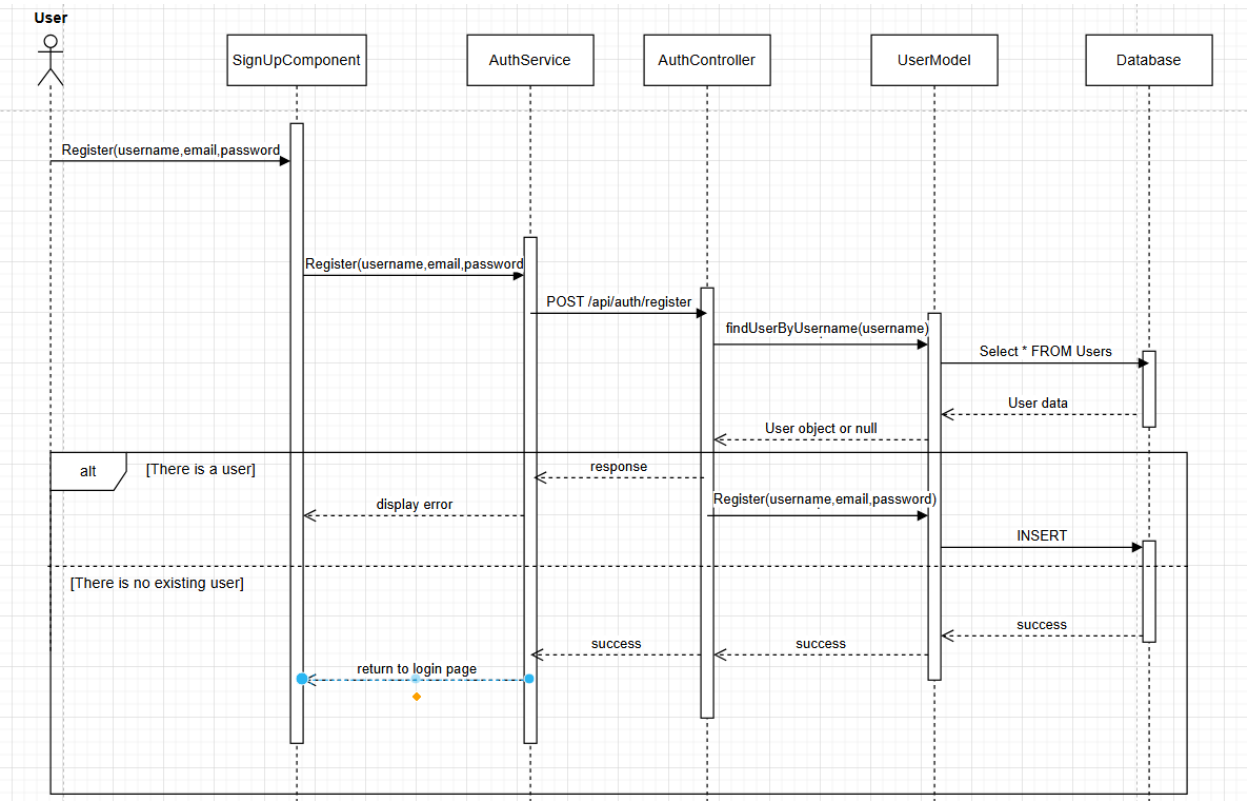


Figure 4. Register Sequence Diagram

c. Sequence Diagram 3: Browsing Lesson

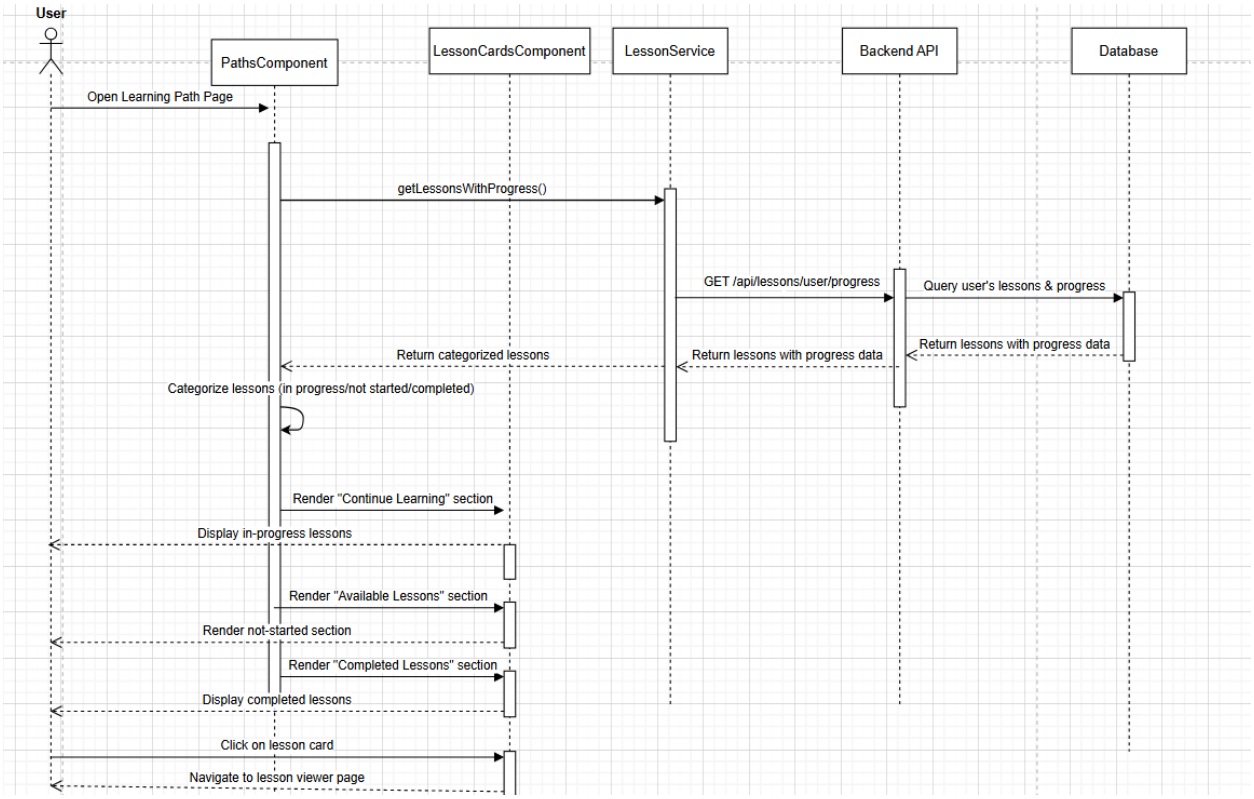


Figure 5. Browsing Lesson Sequence Diagram

d. Sequence Diagram 4: Completing Lesson

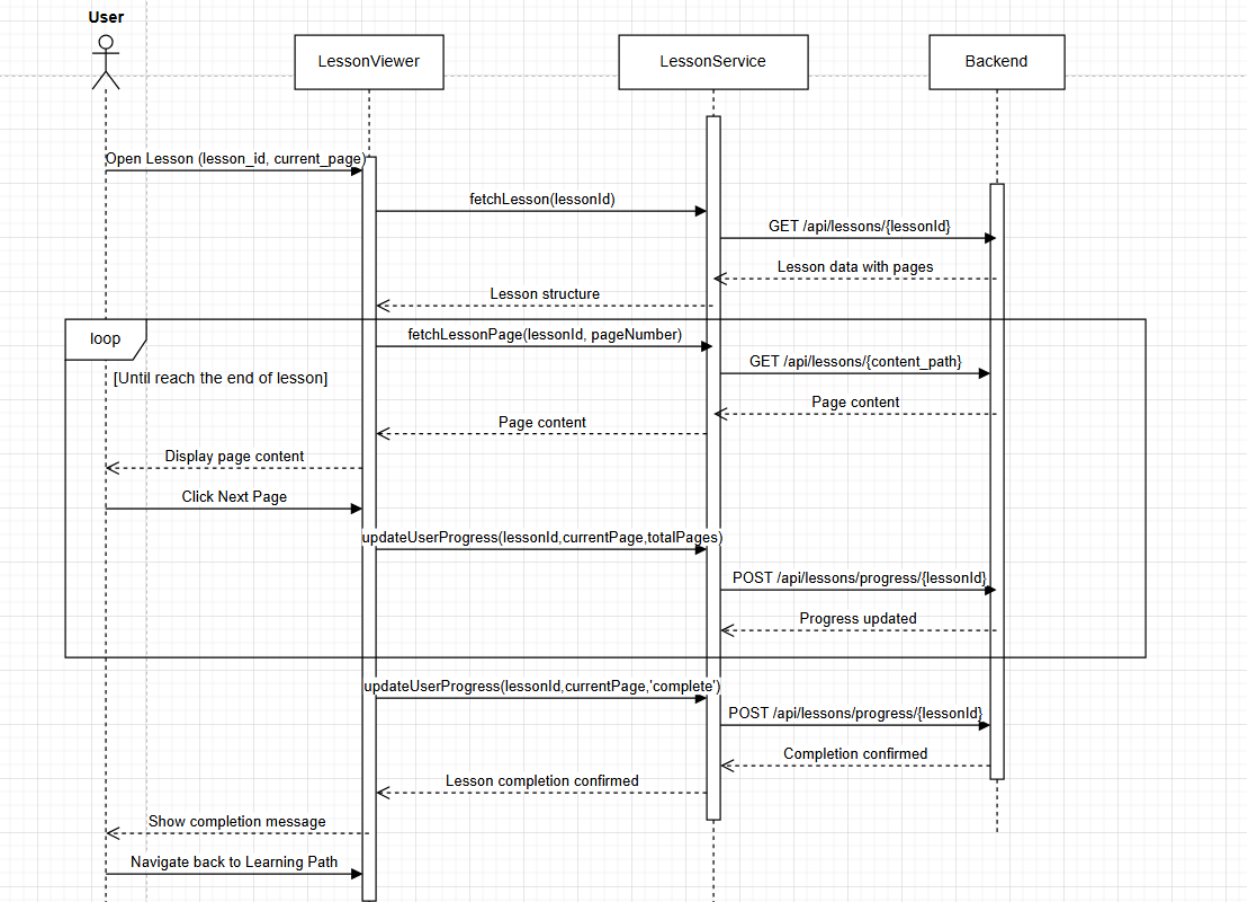


Figure 6. Finish Lesson Sequence Diagram

e. Sequence Diagram 5: Browsing Labs

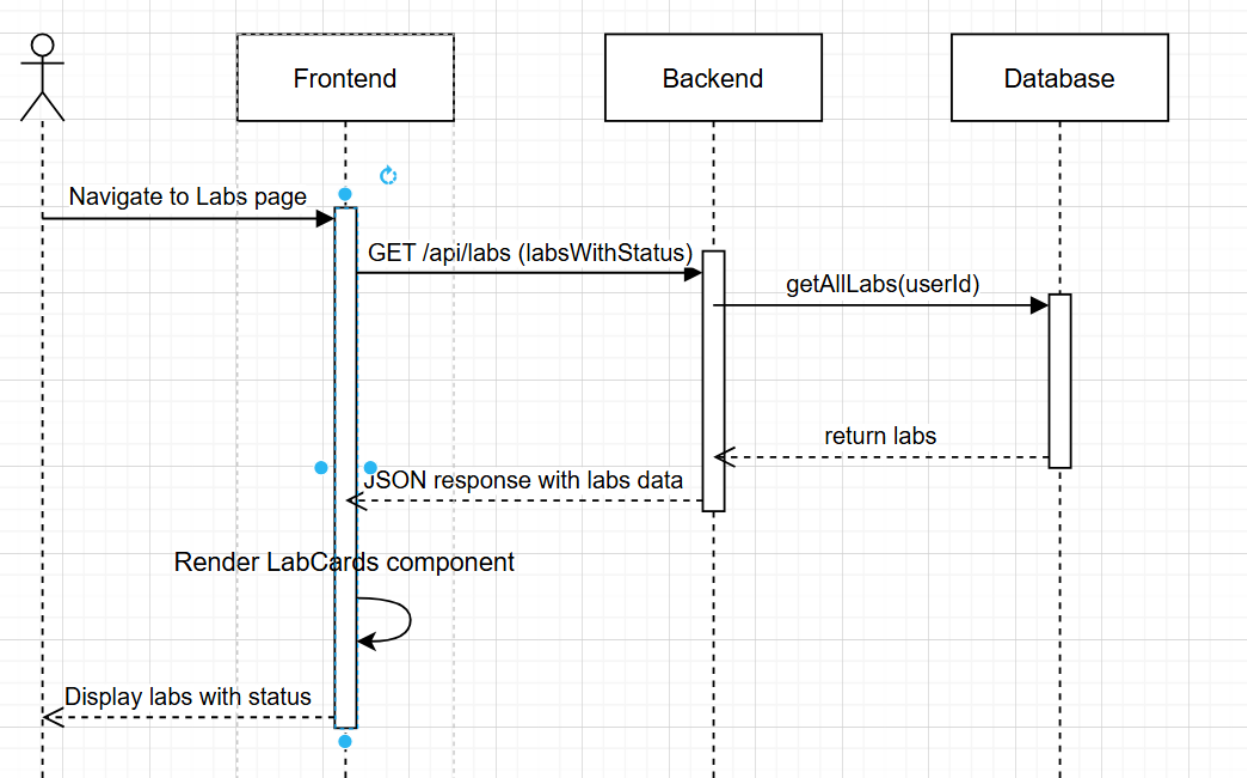


Figure 7. Browsing Labs Sequence Diagram

f. Sequence Diagram: Starting Labs

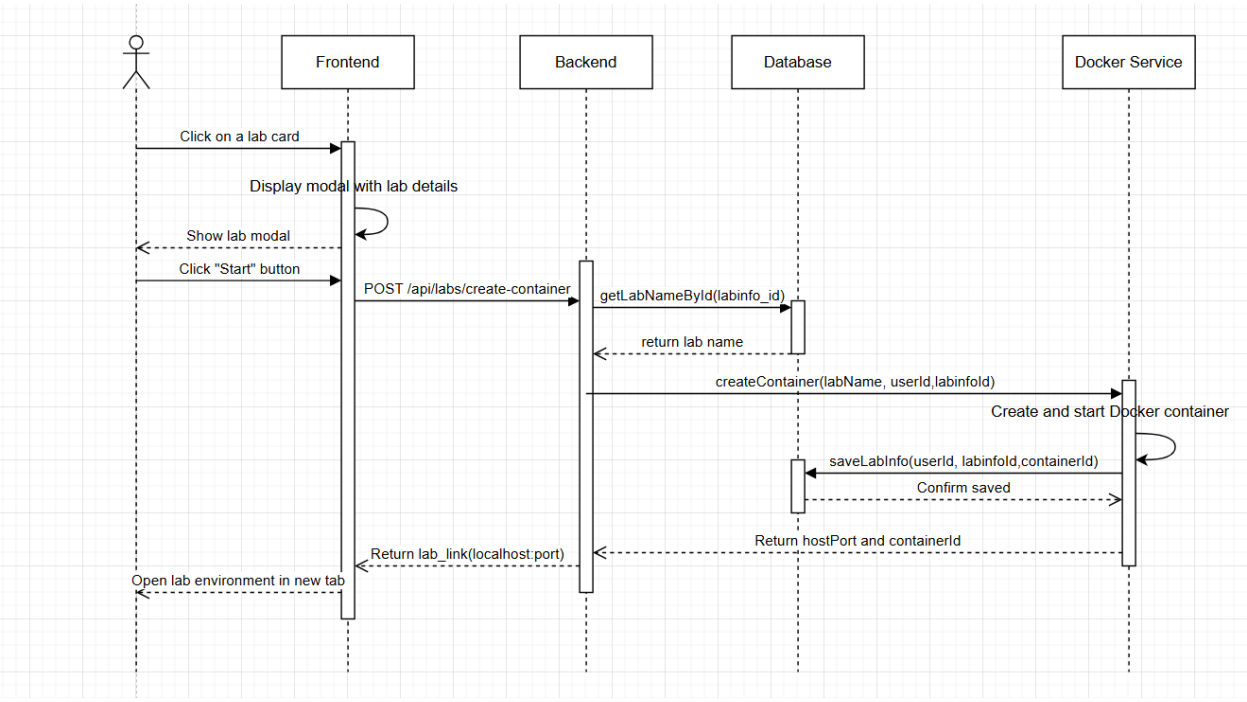


Figure 8. Starting Labs Sequence Diagram

g. Sequence Diagram 7: Submiting Flags

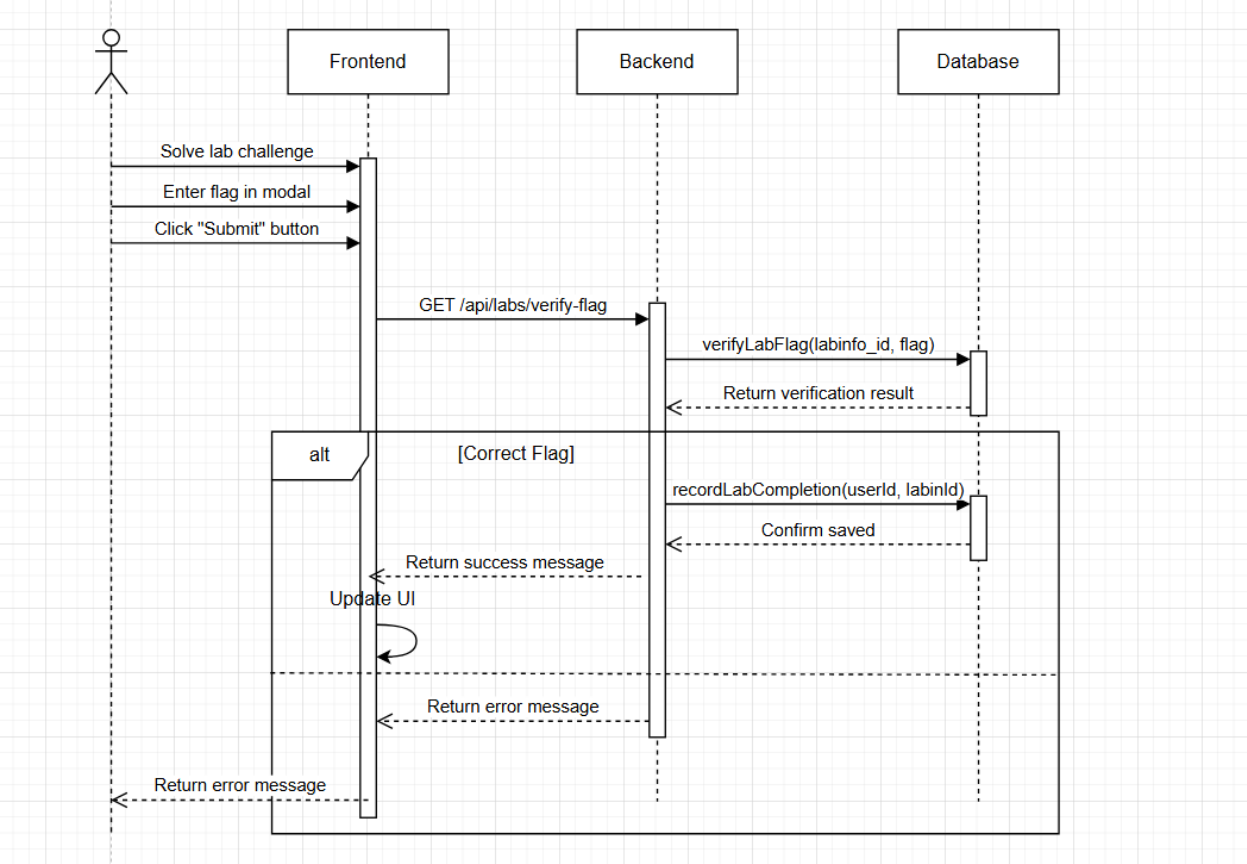


Figure 9. Submitting Flags Sequence Diagram

h. Sequence Diagram 8: Stopping Labs

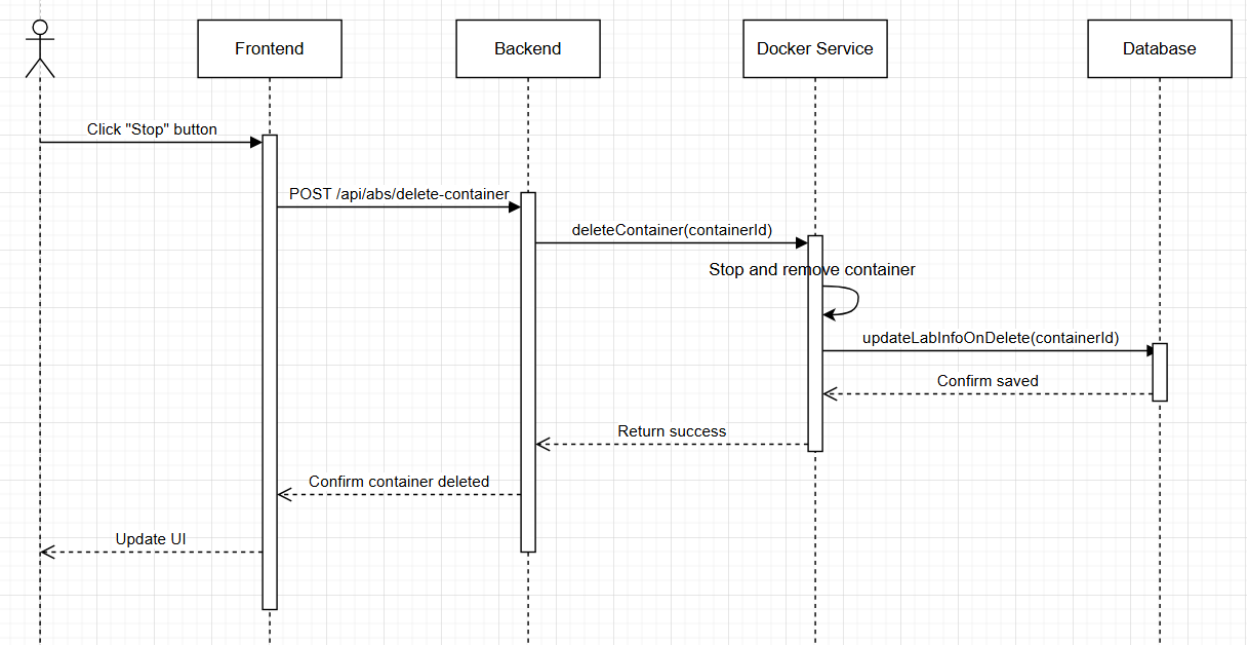


Figure 10. Stopping Labs Sequence Diagram

i. Sequence Diagram 9: Managing Account

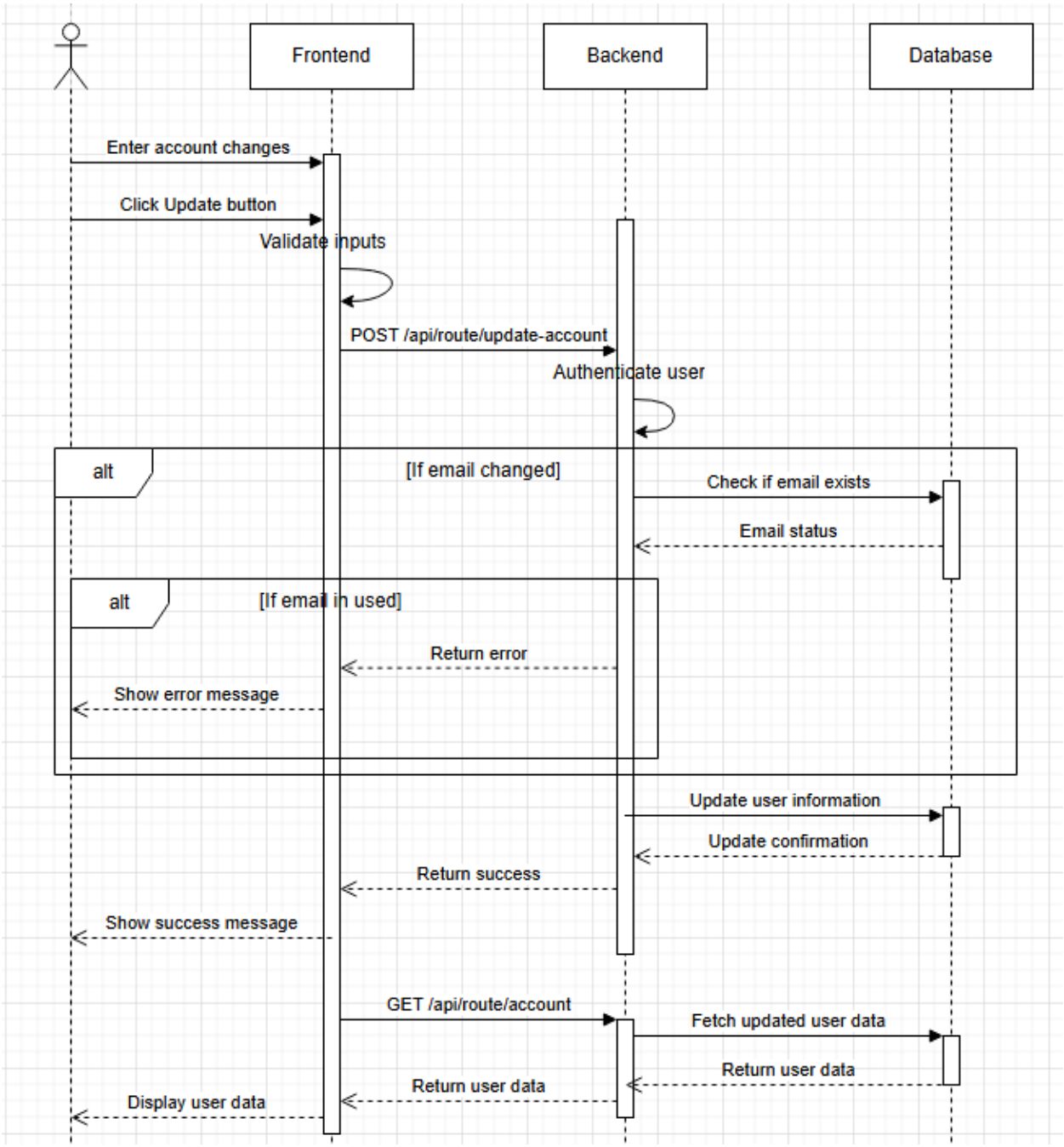


Figure 11. Managing Account Sequence Diagram

j. Sequence Diagram 10: Creating Lessons

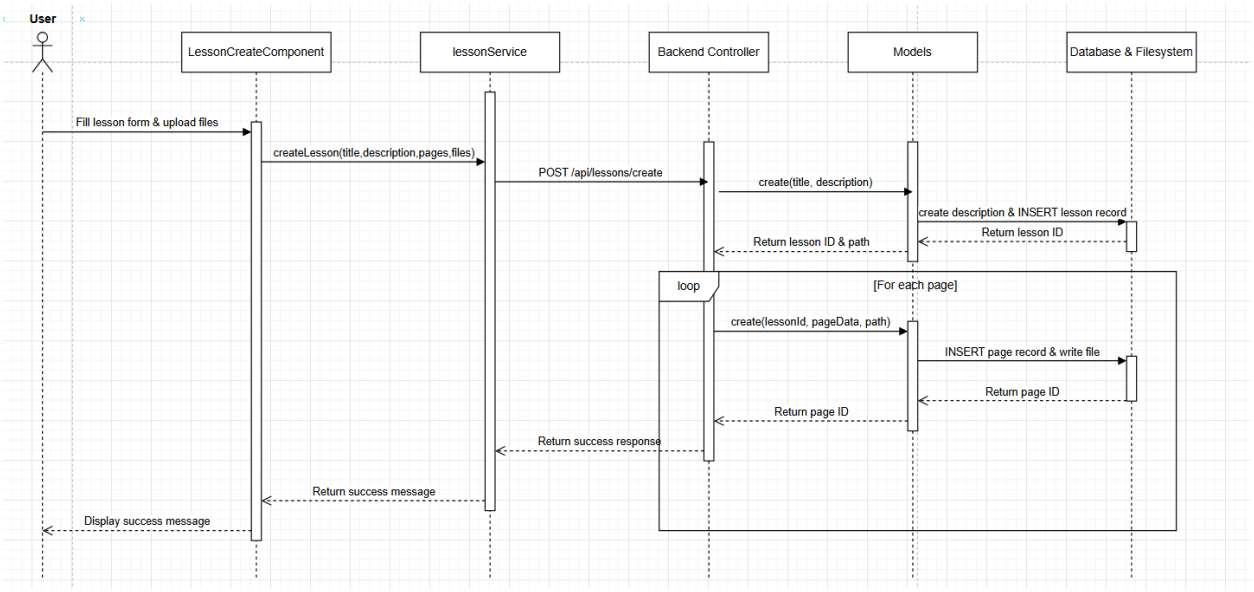


Figure 12. Creating Lessons Sequence Diagram

k. Sequence Diagram 11: Editing Lessons

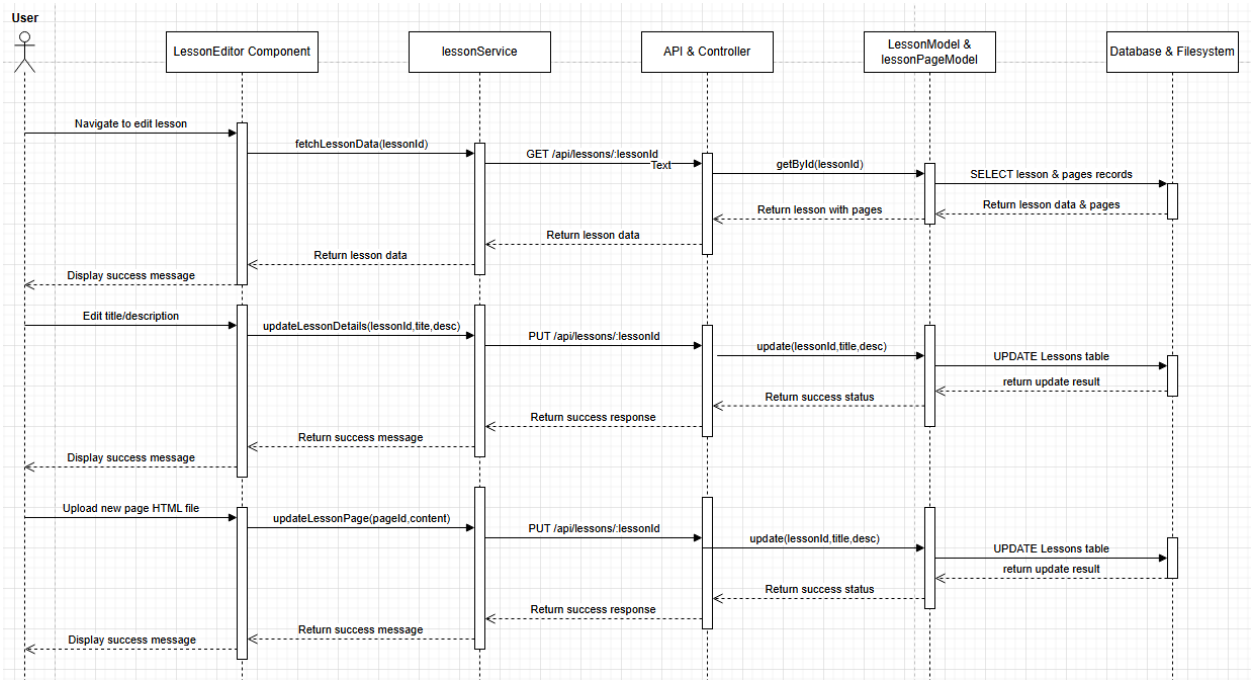


Figure 13. Editing Lessons Sequence Diagram

l. Sequence Diagram 12: Reset Password

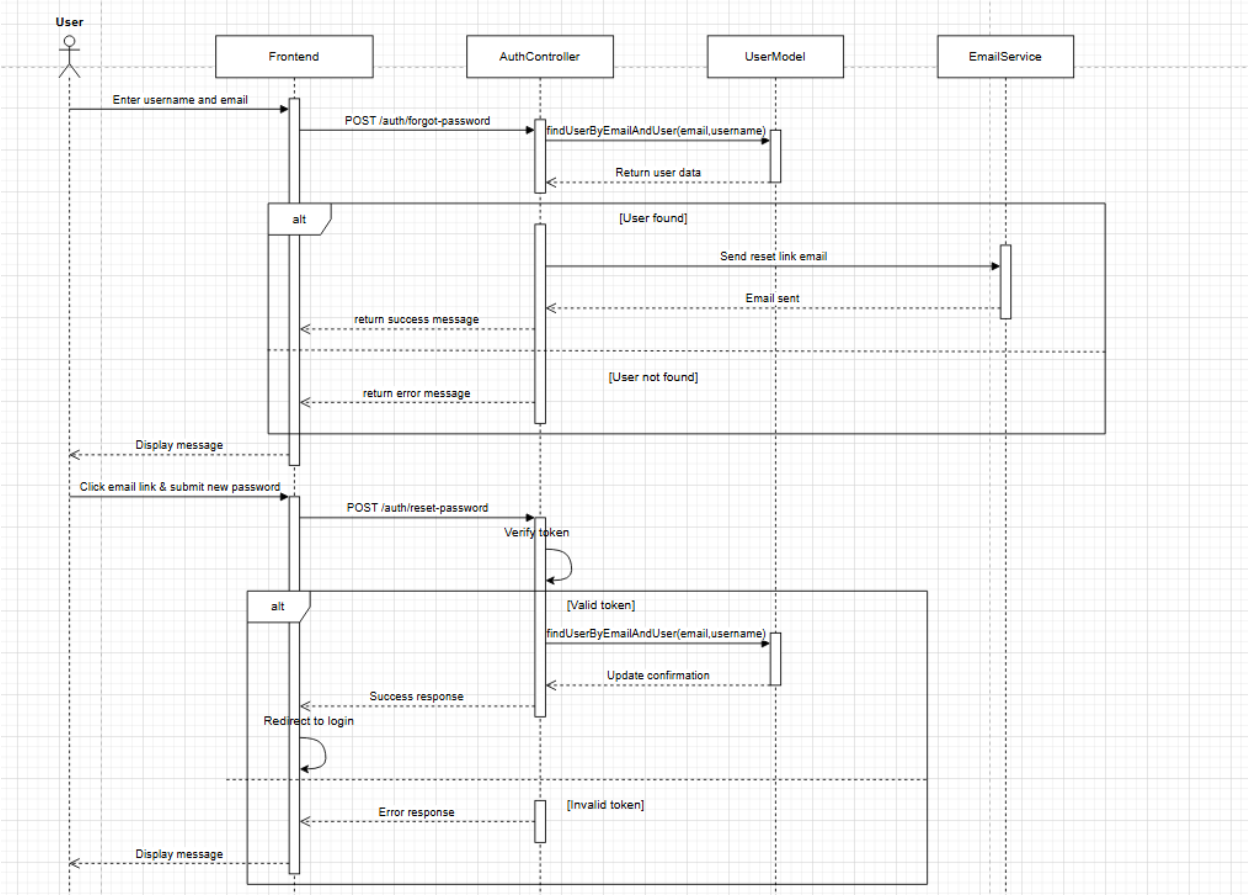


Figure 14. Reset Password Sequence Diagram

5 Implementation

5.1 Authenticate Section:

a. Login:

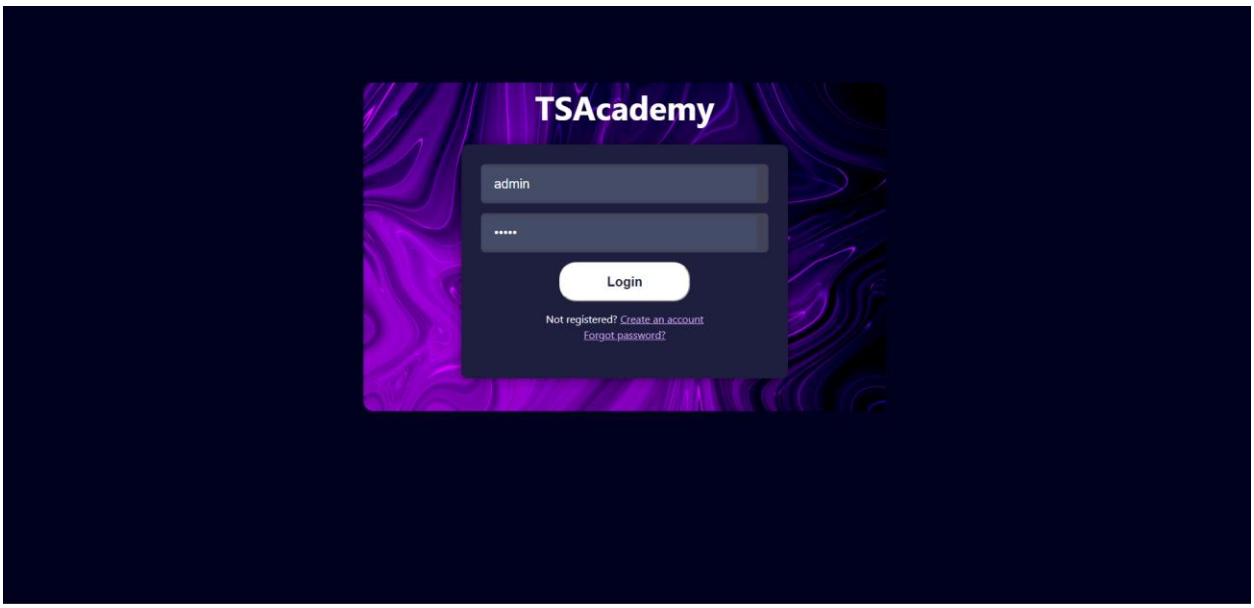


Figure 15. Login Form

If the credential is correct, user will be redirect to homepage.

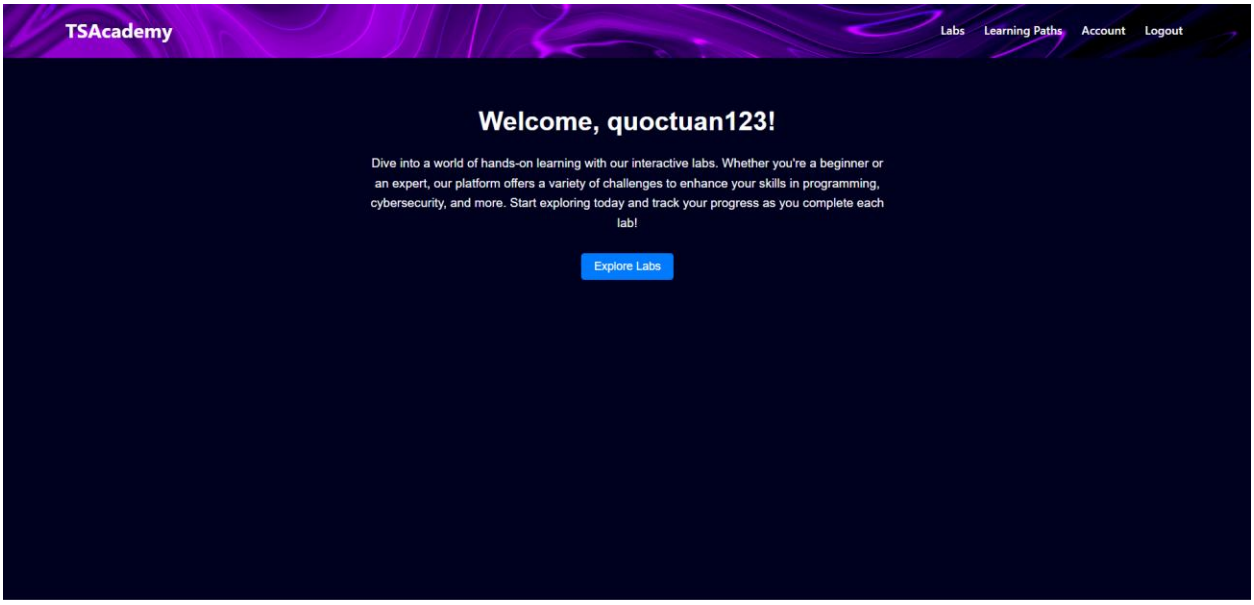


Figure 16. Login Successful

b. Register Account

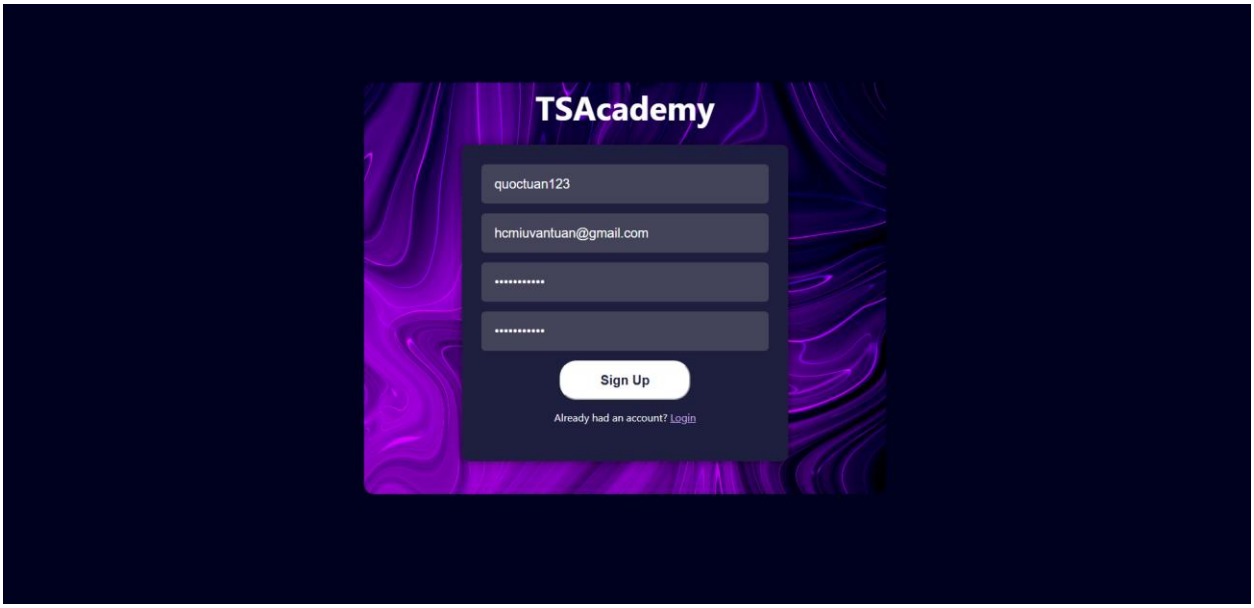


Figure 17. Register Form

If there is not an existing account, it will display success message and redirect to login page.

c. Reset Password

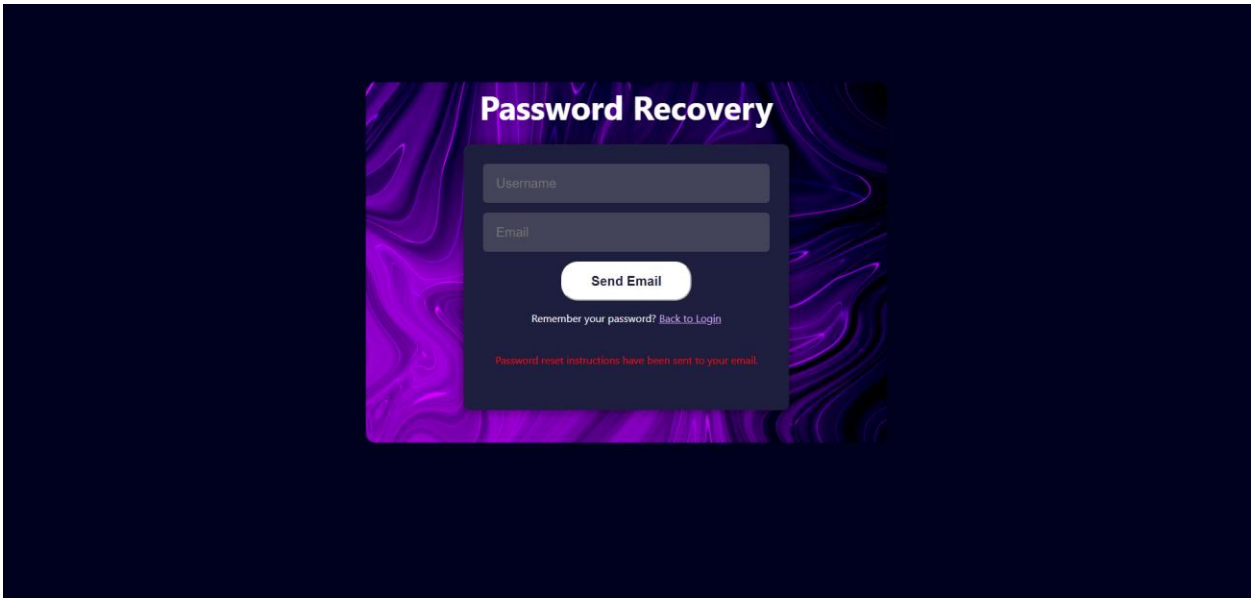


Figure 18. Register Successful

After submitting the right username and email, the server will send a email with a link to reset the password account.

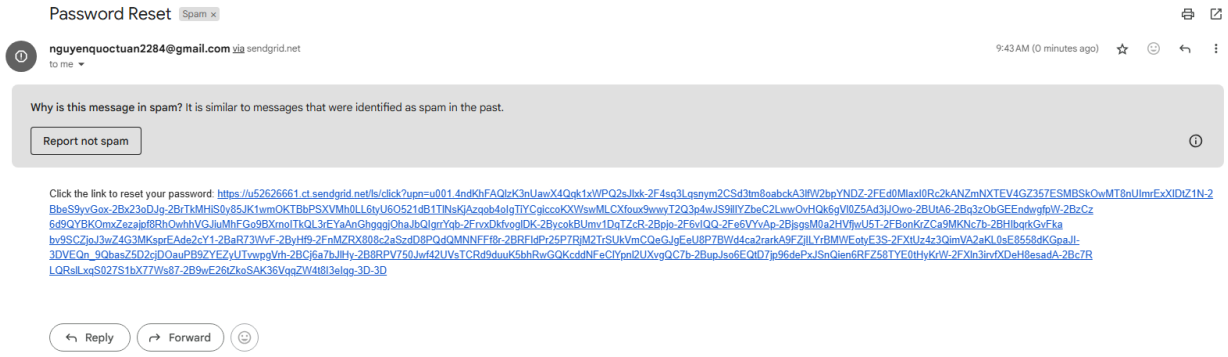


Figure 19. Reset Password Email

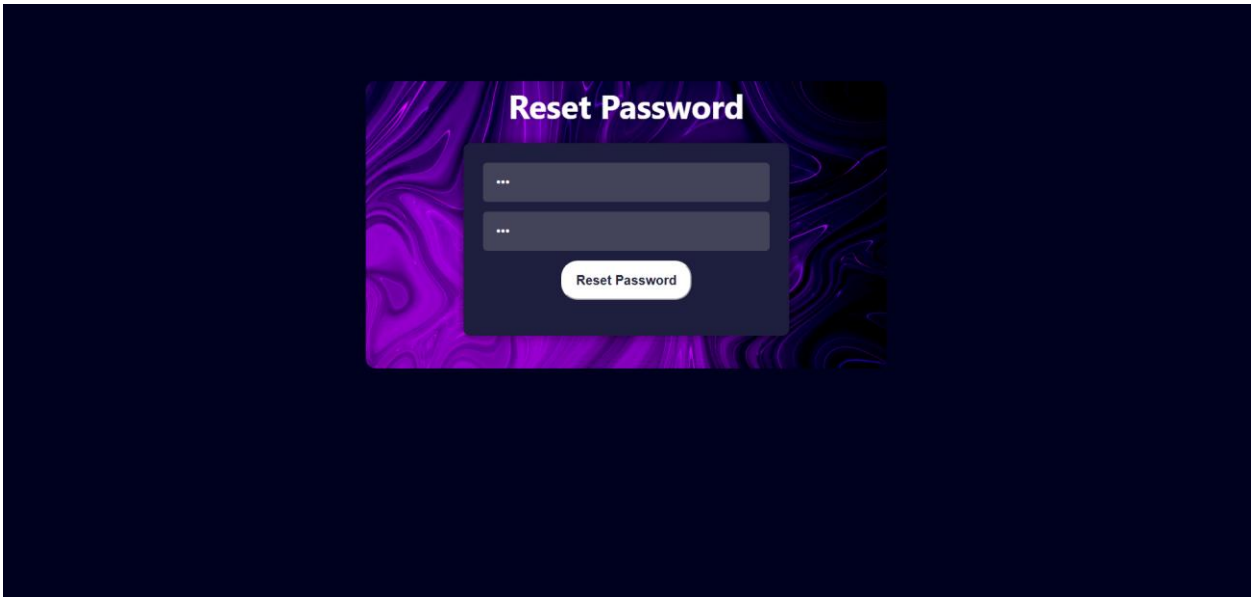


Figure 20. Reset Password Form

Then user will submit the new password then it will redirect to login page.

5.2 Labs Section:

a. Browsing lab:

After click Labs on the Navigate Bar:

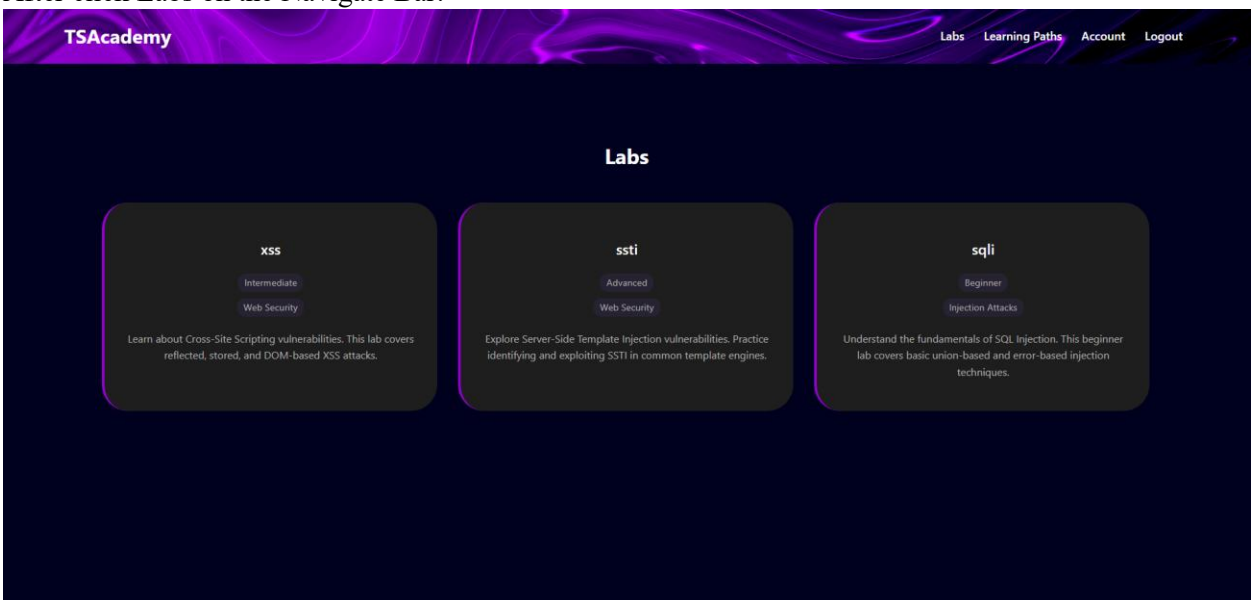


Figure 21. Labs Section

b. Selecting and Finishing Lab

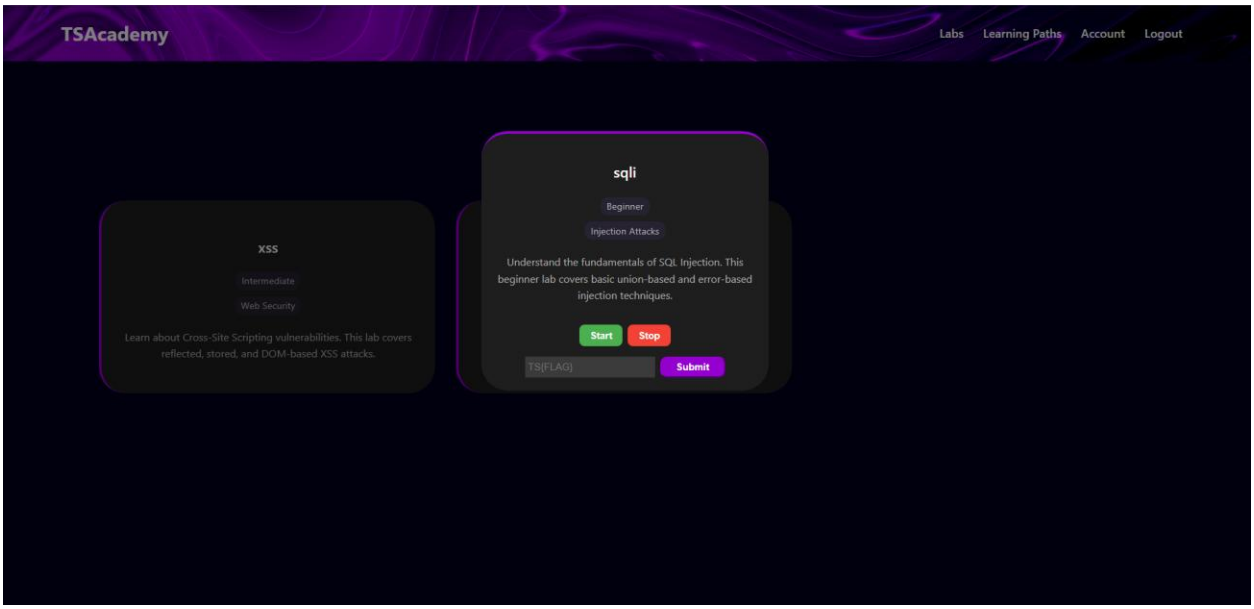


Figure 22. A Lab Card

Clicking one of the labs then press start.

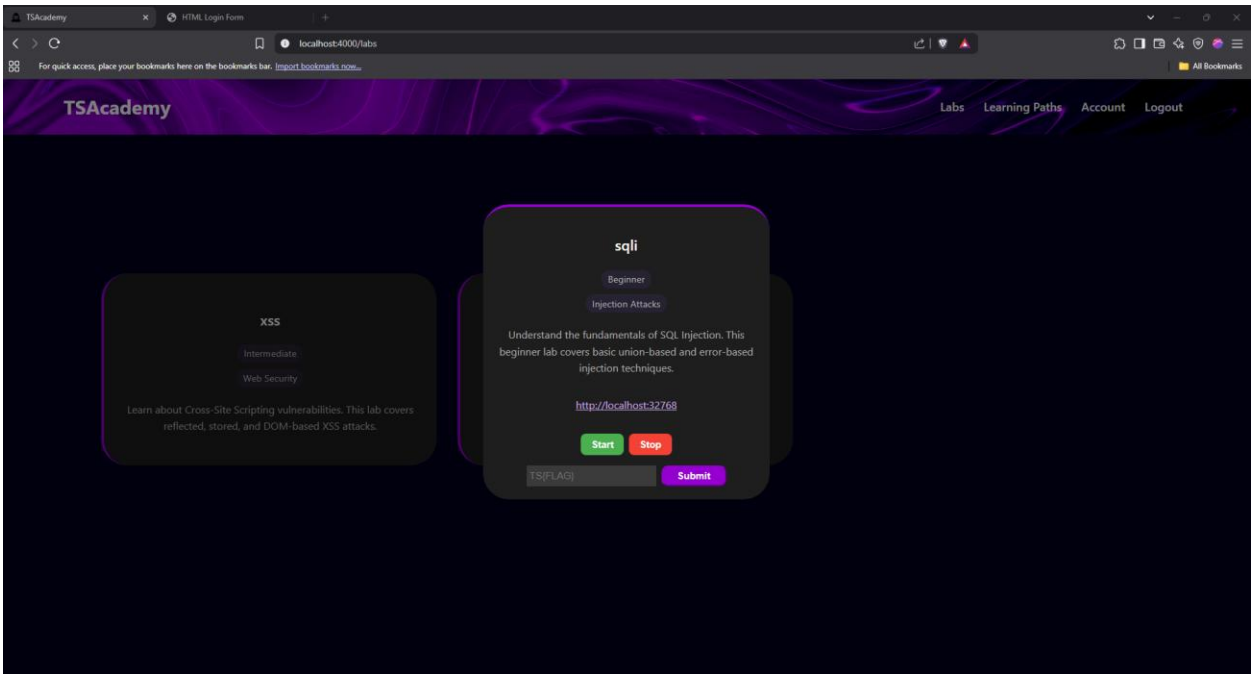


Figure 23. Starting a Lab

The app will open a new window the url of the lab and also display the url of the lab in the lab card.

The lab will stop after clicking stop button or will automatically stop after a period of time.

After solving the lab, submit the flag into the input area then submit.

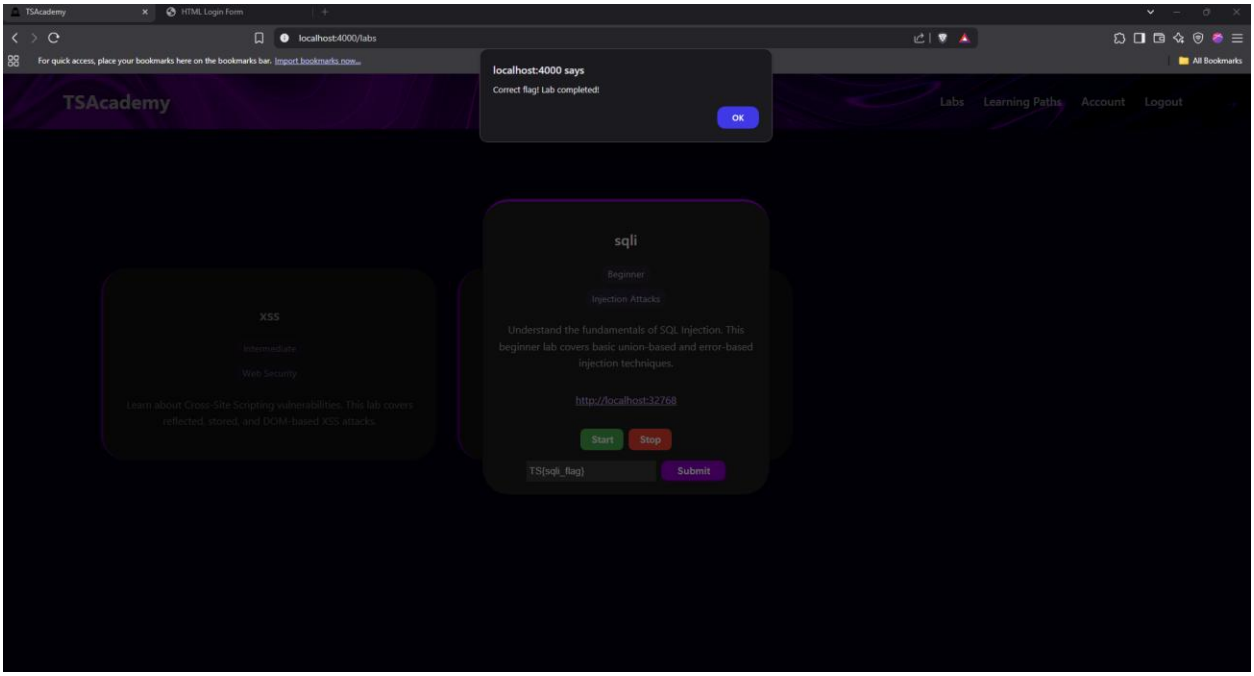


Figure 24. Submitting Flag

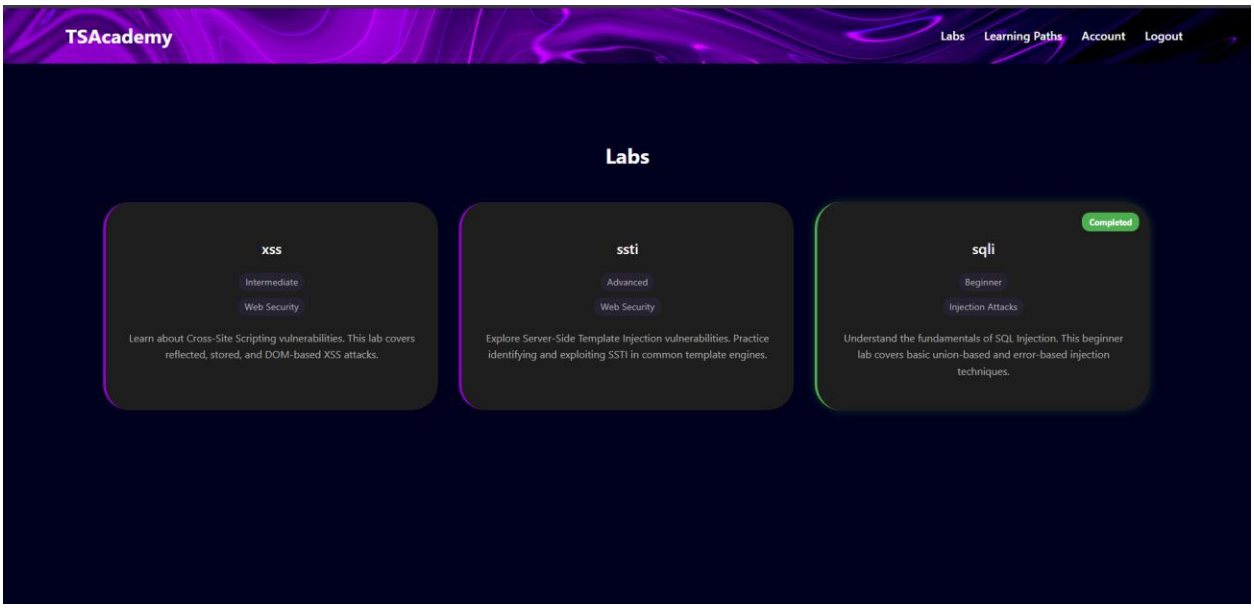


Figure 25. A completed Lab

5.3 Lessons Section:

a. Browsing Lessons:

After clicking on Learning Paths on the navigate bar

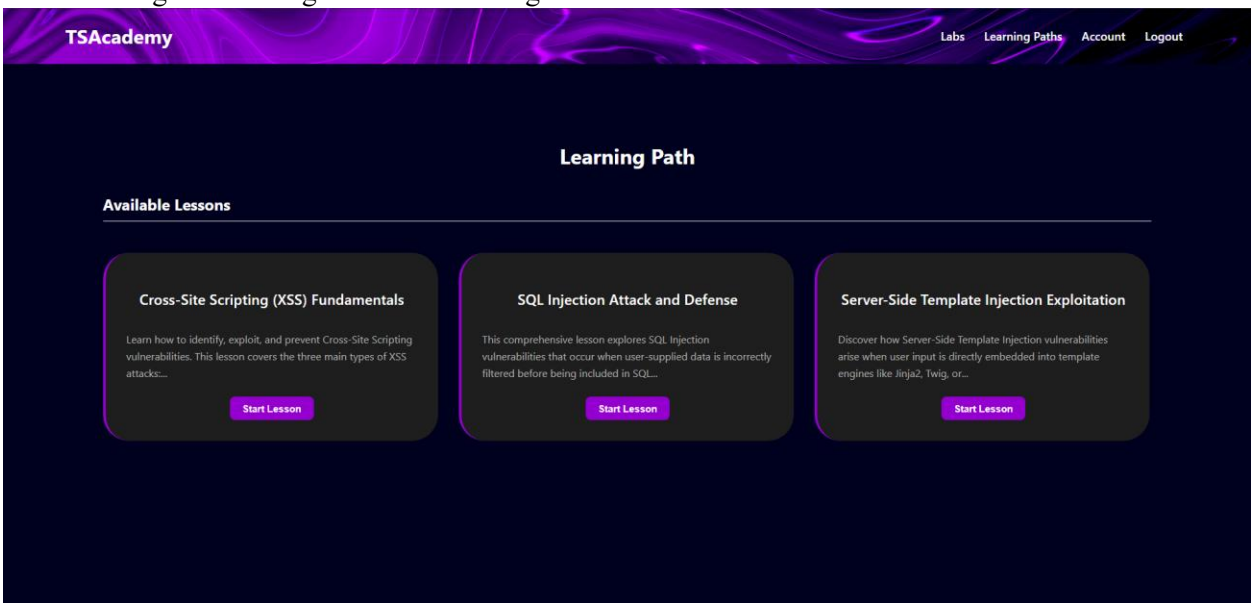


Figure 26. Lessons Section

b. Starting Lessons

Start one of the lessons



Figure 27. An example page

Click next at the end of the page to move to the next page. This also record the progress of the user for the next time access the lesson, it will start at the page were left at.

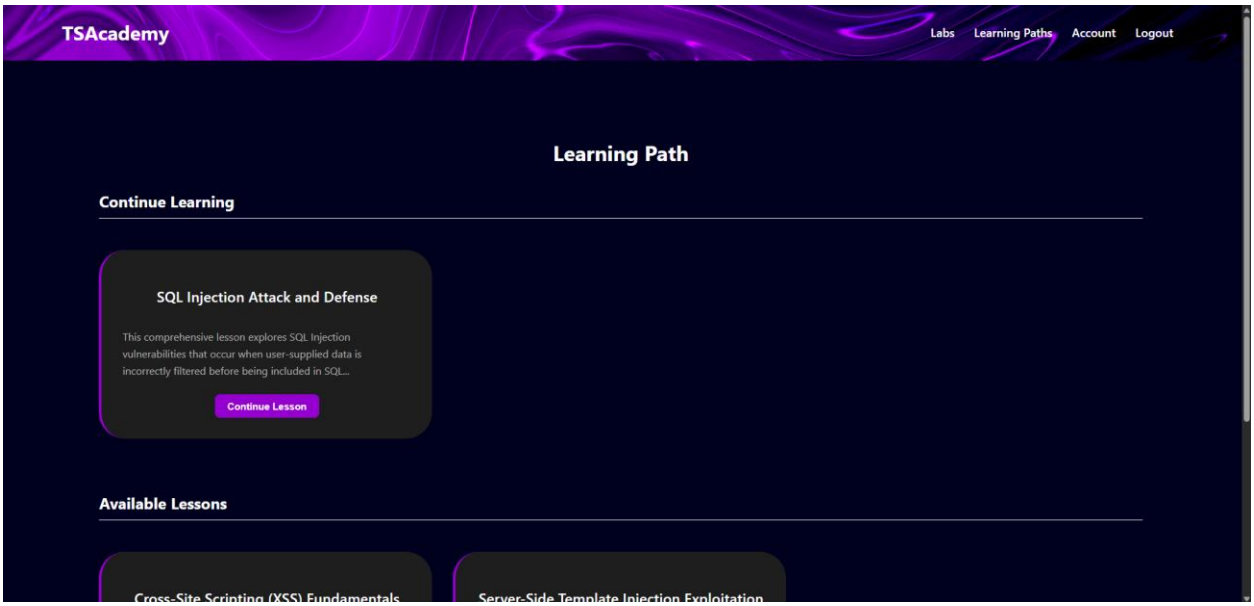


Figure 28. A Unfinished Lesson

Complete the lesson by pressing complete button at the end of the lesson.

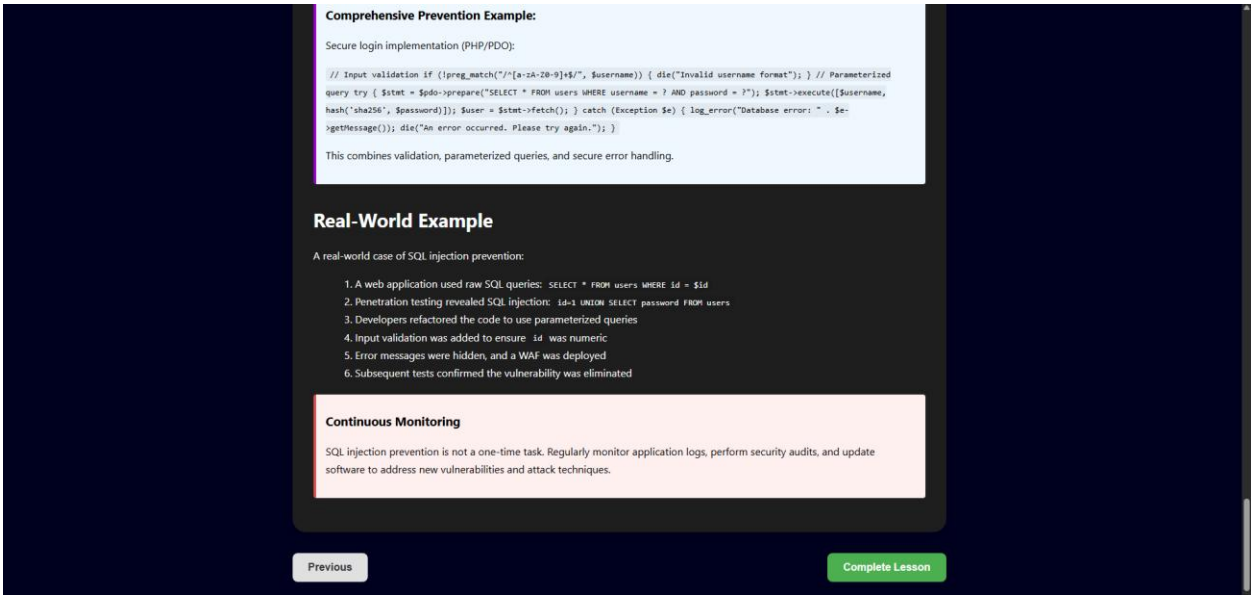


Figure 29. Completing a Lesson

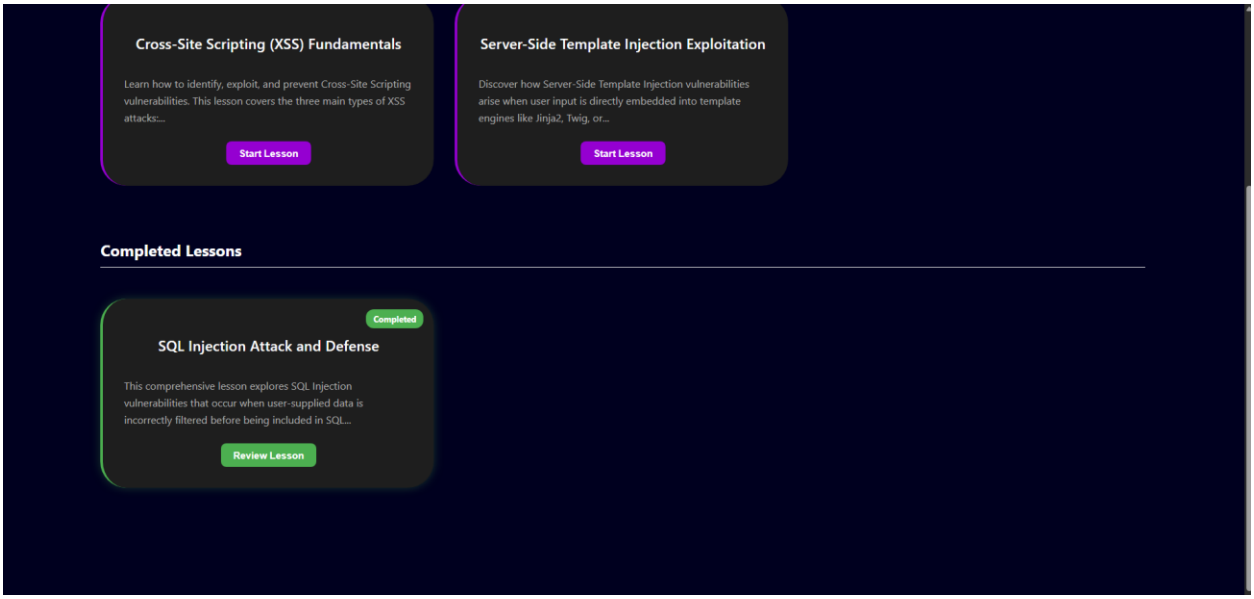


Figure 30. A completed Lesson

5.4 Account Section

a. Updating account

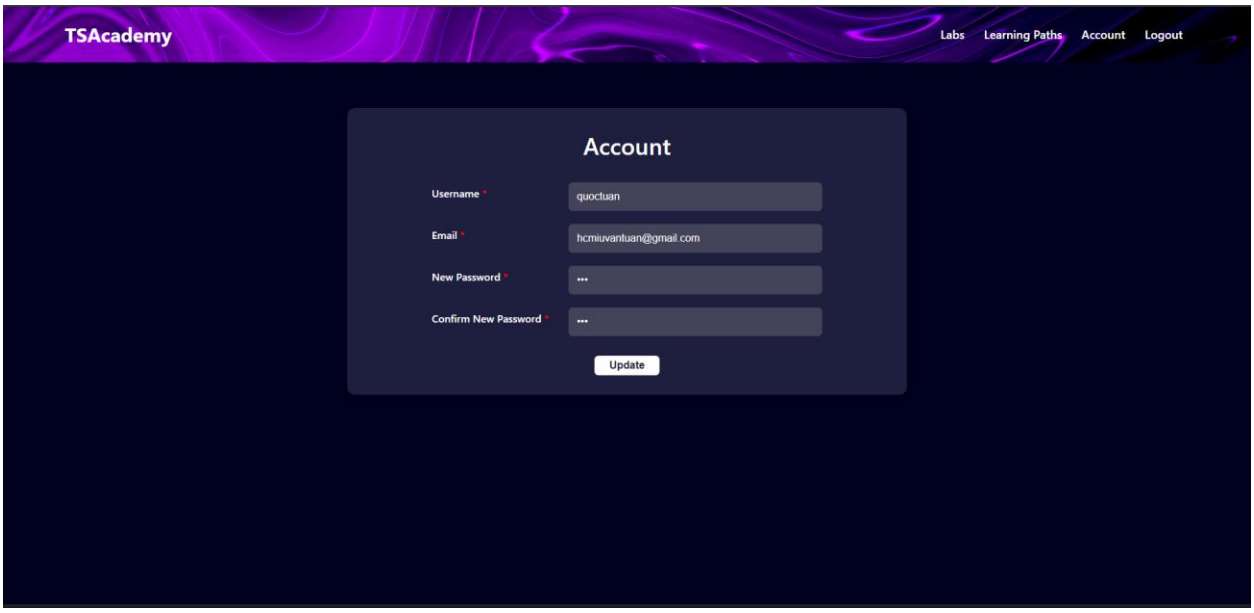


Figure 31.Editting Account Form

Input the value that needs to be update then click update.

5.5 Admin Section

After login as admin account:

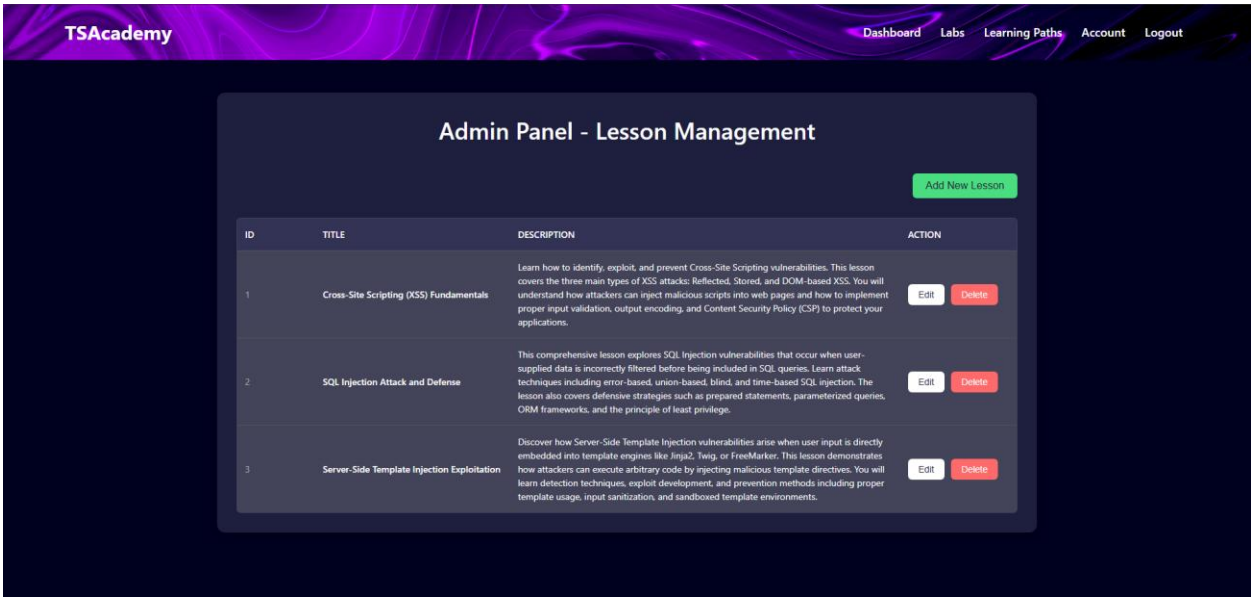


Figure 32. List of Lessons

a. Creating Lesson:

Input title, description.

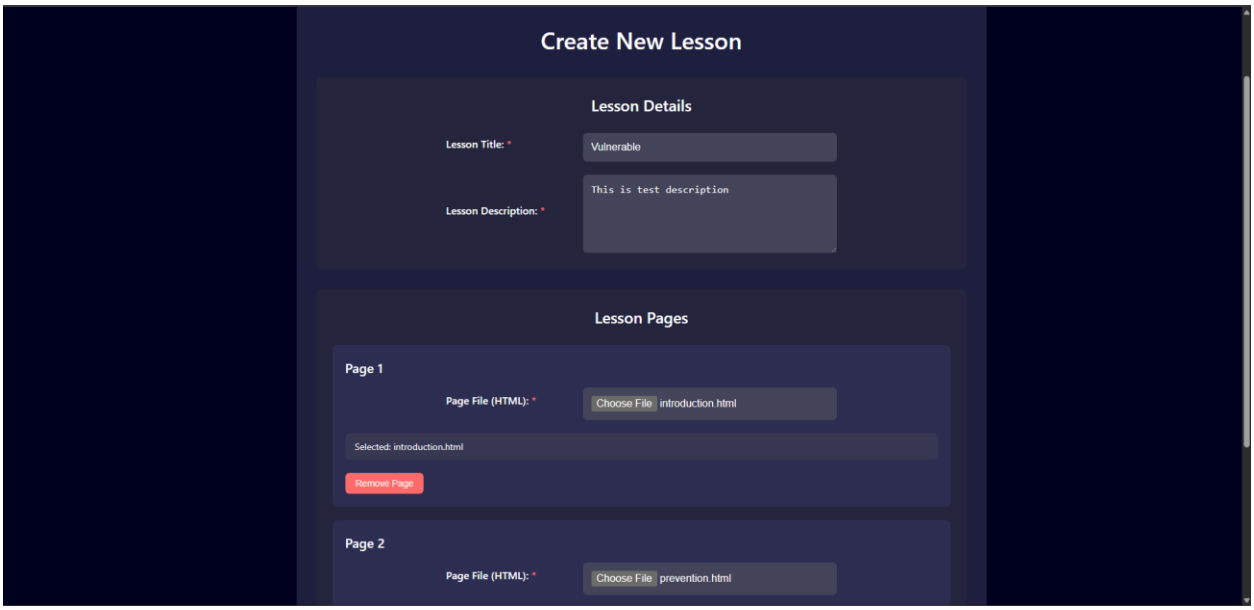


Figure 33. General Information of a Lesson

Upload html for each page

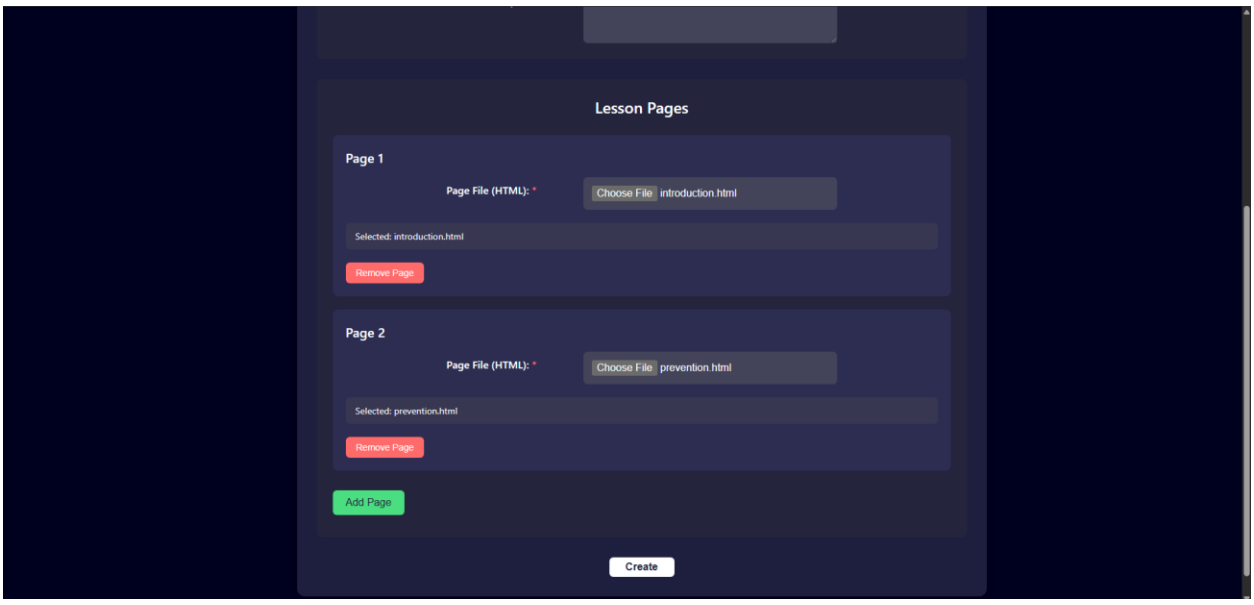


Figure 34. Pages of Lesson Section

After finish upload files, click Create. Then the server will display a success message. Now click back on Dashboard, the new lesson will appear.

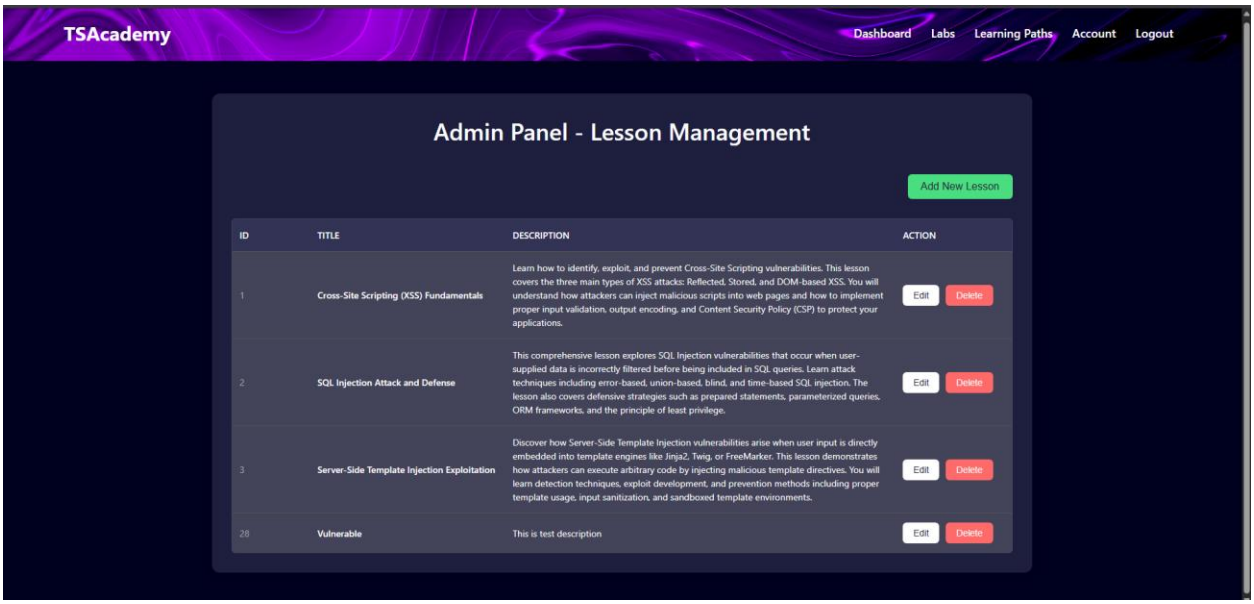


Figure 35. Creating a Lesson successful

b. Editing Lessons

Click edit in one of the lessons

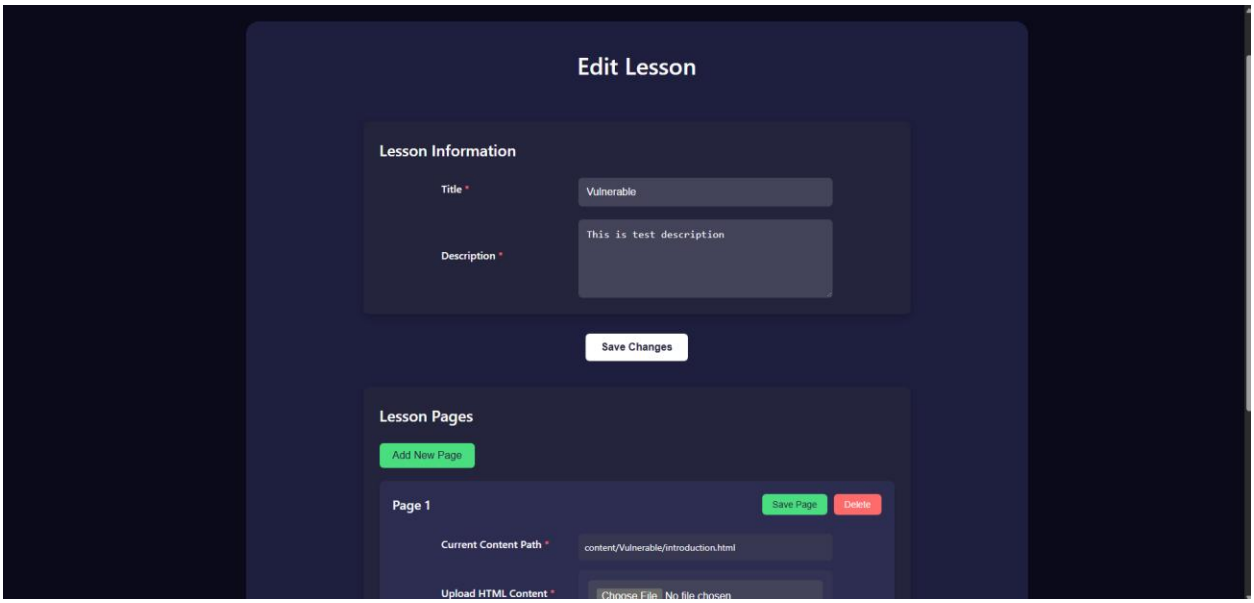


Figure 36. Editting Lesson Section

Changing the Lesson Information then click Save Changes to change the title and description.

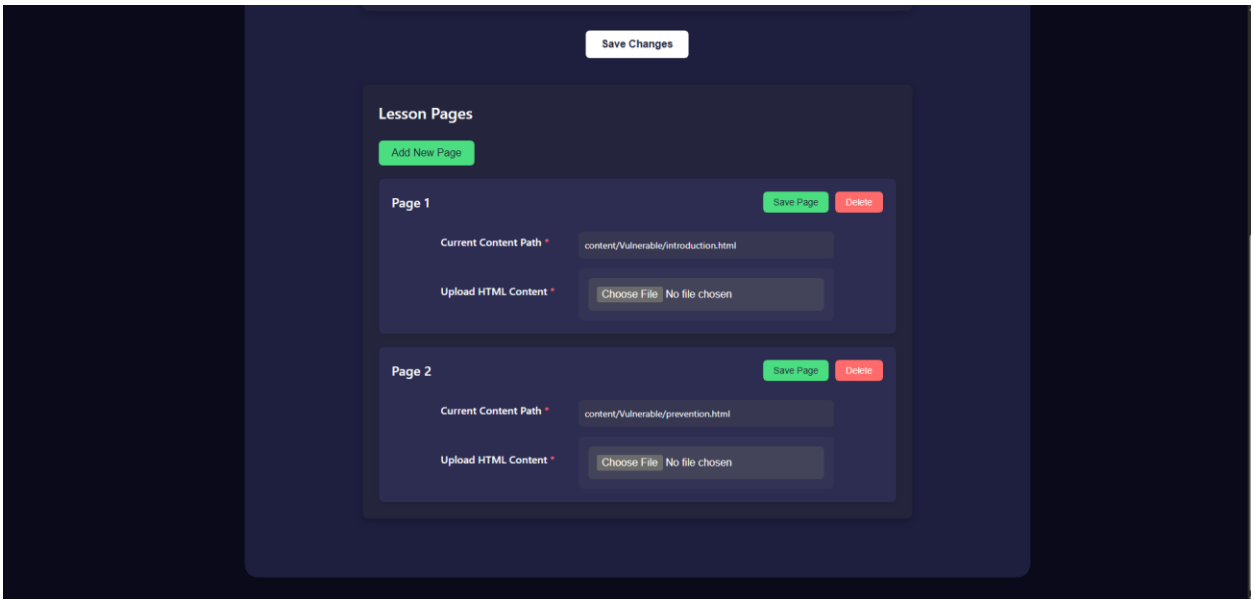


Figure 37. Editting Pages

Adding new page or change the existing pages by upload new html file. Then press Save Page for each page.

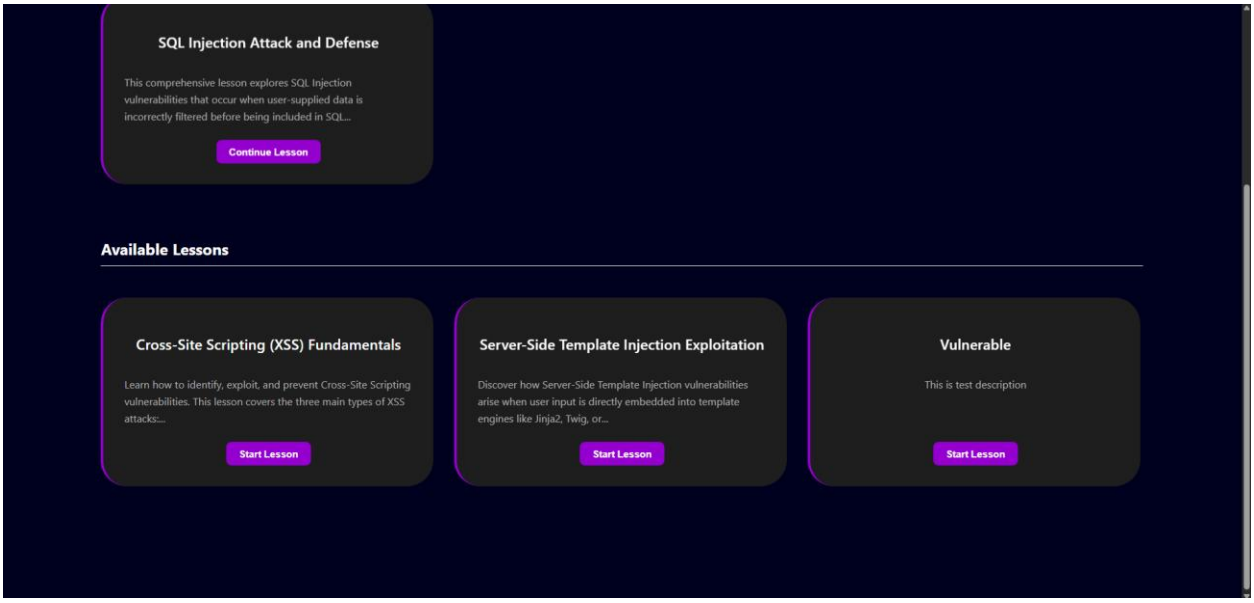


Figure 38. Editting Lesson successful

Checking the new lesson

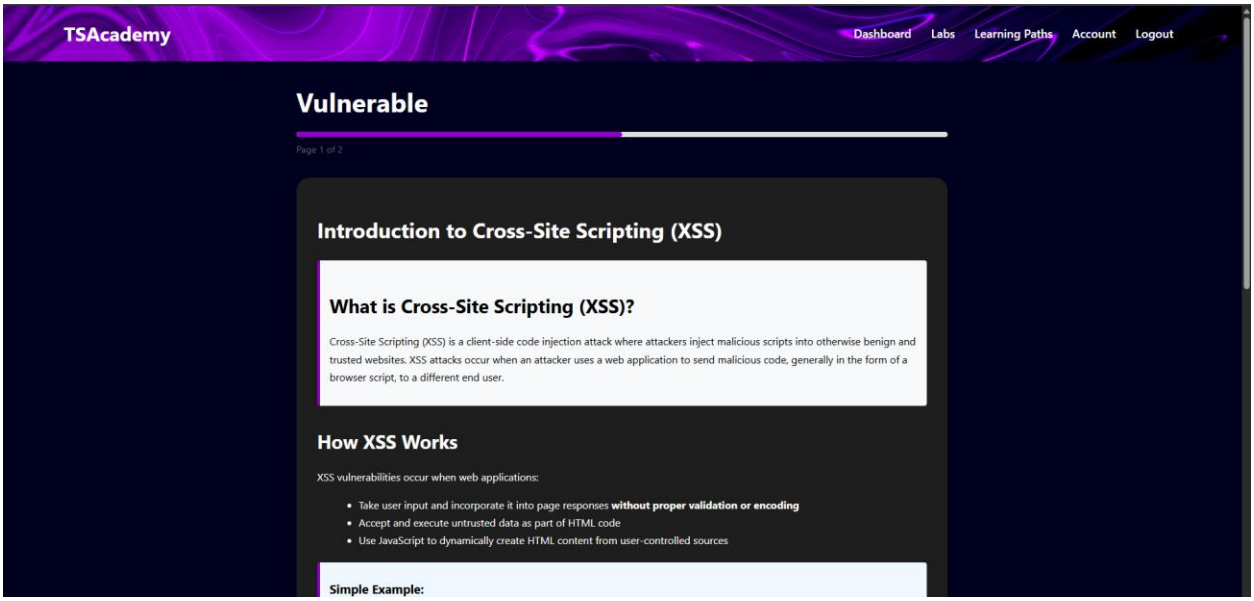


Figure 39. Editted Lesson

6 Discussion and conclusion

This project aimed to develop a comprehensive online learning platform focused specifically on cybersecurity education. In today's increasingly digital world, the demand for cybersecurity professionals continues to grow exponentially, creating a significant skills gap in the industry. Our platform represents a strategic response to this challenge by providing accessible, interactive, and up-to-date cybersecurity training content for learners at various skill levels. The web application leverages modern technologies and pedagogical approaches to create an engaging learning environment that prepares users for real-world cybersecurity scenarios.

6.1 Challenges Encountered

Despite careful planning, several challenges emerged during development. The creation of realistic yet controlled sandbox environments for practical exercises presented technical difficulties, particularly in balancing system security with the need for realistic training scenarios. Ensuring content remained current in the rapidly evolving cybersecurity landscape also proved challenging. Our solution involved implementing an agile content management system that facilitates regular updates.

6.2 Future Work

Expanding the range of specialized topics would increase the platform's utility for professionals seeking to develop expertise in specific security domains. Incorporating emerging technologies such as augmented reality could further enhance the realism and effectiveness of practical training scenarios.

a. AI-Powered Personalized Feedback Systems

A significant planned enhancement is the integration of artificial intelligence to provide personalized feedback for each user. This AI-driven system would analyze user interactions, code submissions, and exercise completions to offer tailored guidance beyond what predetermined response patterns can provide. The system could identify specific misconceptions, suggest alternative approaches to problems,

and highlight areas for improvement in a learner's security methodology. This personalized feedback loop would substantially enhance the educational experience by mimicking the individualized attention of a human instructor while scaling across thousands of users simultaneously.

b. Advanced Laboratory Environments

Future work includes developing an expanded suite of specialized laboratory environments that simulate diverse cybersecurity scenarios with greater fidelity. These enhanced labs would include:

- Advanced Penetration Testing Environments: Configurable network topologies with various vulnerabilities that adapt based on user skill level
- Security Operations Center (SOC) Simulation: Real-time threat detection and incident response scenarios using industry-standard tools
- Secure DevOps Pipeline Labs: Environments to practice implementing security throughout the software development lifecycle
- Threat Hunting Workshops: Dynamic environments populated with subtle indicators of compromise for users to discover
- Forensic Analysis Laboratories: Comprehensive digital forensics scenarios with realistic artifacts and evidence chains

References

- Portswigger. (2018). *Web Application Security, Testing, & Scanning*. Portswigger.net. https://portswigger.net/
- *Node.js v20.2.0 Documentation*. (n.d.). Nodejs.org. https://nodejs.org/docs/latest/api/
- Docker. (2019, October 31). *Docker Documentation*. Docker Documentation. https://docs.docker.com/
- GeeksforGeeks. (2022, March 21). *MVC Framework Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/mvc-framework-introduction/
- jwt. (2024). *JWT.IO - JSON Web Tokens Introduction*. Jwt.io. https://jwt.io/introduction
- Meta Open Source. (2025). *React*. React.dev. https://react.dev/
- Node.js Tutorial for Beginners: Learn Node in 1 Hour | Mosh. (2018). [YouTube Video]. In *YouTube*. https://www.youtube.com/watch?v=TlB_eWDSMt4
- Programming with Mosh. (2023, March 12). *React Tutorial for Beginners*. Www.youtube.com. https://www.youtube.com/watch?v=SqcY0GlETPk
- *MySQL :: MySQL 8.4 Reference Manual*. (n.d.). Dev.mysql.com. https://dev.mysql.com/doc/refman/8.4/en/
- *SendGrid | SendGrid Docs | Twilio*. (2024). Twilio.com. https://www.twilio.com/docs/sendgrid